МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика» **Кафедра** 806

Группа М80-407Б-22 **Направление подготовки** 01.03.02 «Прикладная математика и информатика»

Профиль Информатика

Квалификация: бакалавр

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

На тему: Открытая микросервисная платформа для автоматизации тренировочного процесса и взаимодействия в системе «тренер-спортсмен»

Автор ВКРБ: Калуцкий Максим Витальевич (_____)

Руководитель: Булакина Мария Борисовна (_____)

Рецензент: Булакина Мария Борисовна (_____)

К защите допустить

Заведующий кафедрой № 806 Крылов Сергей Сергеевич (_____)

____ мая 2026 года

Москва 2026

РЕФЕРАТ

Выпускная квалификационная работа бакалавра состоит из 73 страниц, 7 рисунков, 20 таблиц, 12 использованных источников, 8 приложений.

BACKEND-ПЛАТФОРМА, МИКРОСЕРВИСНАЯ АРХИТЕКТУРА, ТРЕНЕР, СПОРТСМЕН, ТРЕНИРОВОЧНЫЙ ПРОЦЕСС, REST API, NATS JETSTREAM, POSTGRESQL, АНАЛИТИКА, AI-РЕКОМЕНДАЦИИ

Выпускная квалификационная работа посвящена разработке backend-части открытой микросервисной платформы CoachLink для стандартизации и автоматизации взаимодействия в системе «тренер-спортсмен». В работе рассматривается проблема разрозненного обмена тренировочными заданиями и отчетами через мессенджеры, электронные таблицы и произвольные форматы данных.

Объектом исследования является процесс информационного взаимодействия между тренером и спортсменом при планировании, выполнении и анализе тренировок. Предметом исследования является backend-платформа для автоматизации назначения тренировочных заданий, сбора стандартизированных отчетов, отправки уведомлений и анализа данных тренировочного процесса.

Целью работы является разработка открытой микросервисной backend-платформы CoachLink для стандартизации и автоматизации взаимодействия в системе «тренер-спортсмен».

В рамках работы спроектирована микросервисная архитектура, включающая API Gateway, Auth Service, User Service, Training Service, Notification Service, Analytics Service и AI Service. Сервисы реализованы на языке Go, используют PostgreSQL для хранения данных, NATS JetStream для событийного взаимодействия и OpenAPI для документирования публичных интерфейсов. Реализованный backend обеспечивает регистрацию пользователей, разграничение ролей тренера и спортсмена, управление связями и тренировочными группами, создание тренировочных планов, назначение тренировок, прием стандартизированных отчетов, формирование уведомлений, агрегацию тренировочных данных и выдачу AI-рекомендаций по последним отчетам спортсмена.

AI-рекомендации формируются через Ollama и рассматриваются как вспомогательный инструмент тренера.

Предметная реализация первой версии ориентирована на легкую атлетику, однако архитектура платформы не ограничивается этим видом спорта и допускает дальнейшее расширение структуры тренировок, отчетов, шаблонов и метрик для других спортивных дисциплин.

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ . . . 7
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ . . . 8
ВВЕДЕНИЕ . . . 9
1 Аналитический обзор . . . 14
1.1 Особенности взаимодействия тренера и спортсмена . . . 14
1.2 Использование мессенджеров . . . 14
1.3 Использование электронных таблиц . . . 15
1.4 Спортивные трекеры и приложения . . . 16
1.5 Закрытые коммерческие платформы . . . 17
1.6 Сравнительный анализ подходов . . . 18
1.7 Требования к разрабатываемой backend-платформе . . . 19
1.7.1 Пользователи и роли . . . 19
1.7.2 Связь тренер-спортсмен . . . 19
1.7.3 Тренировочные группы . . . 20
1.7.4 Тренировочные планы и назначения . . . 20
1.7.5 Отчёты . . . 20
1.7.6 Уведомления . . . 20
1.7.7 Аналитика . . . 20
1.7.8 AI-рекомендации . . . 21
1.8 Нефункциональные требования . . . 21
1.9 Выводы по главе 1 . . . 21
2 Проектная часть . . . 23
2.1 Общий подход к проектированию . . . 23
2.2 Высокоуровневая архитектура . . . 23
2.3 Состав сервисов . . . 24
2.3.1 API Gateway . . . 24
2.3.2 Auth Service . . . 25
2.3.3 User Service . . . 26
2.3.4 Training Service . . . 26
2.3.5 Notification Service . . . 27
2.3.6 Analytics Service . . . 28
2.3.7 AI Service . . . 28
2.4 Модель данных . . . 29

2.4.1 Auth Service . . . 29
2.4.2 User Service . . . 29
2.4.3 Training Service . . . 30
2.4.4 Notification Service . . . 30
2.4.5 ER-диаграмма . . . 31
2.5 Межсервисное взаимодействие . . . 31
2.5.1 Синхронное взаимодействие . . . 31
2.5.2 Асинхронное взаимодействие . . . 32
2.6 Проектирование ключевых сценариев . . . 33
2.6.1 Регистрация пользователя . . . 33
2.6.2 Создание связи тренер-спортсмен . . . 34
2.6.3 Назначение тренировки группе . . . 34
2.6.4 Отправка отчёта . . . 35
2.6.5 Получение AI-рекомендации . . . 35
2.7 Проектирование публичного API . . . 35
2.8 Расширяемость под другие виды спорта . . . 36
2.9 Выводы по главе 2 . . . 37
3 Реализация и тестирование . . . 38
3.1 Общая структура реализации . . . 38
3.2 API Gateway . . . 38
3.3 Auth Service . . . 39
3.3.1 Регистрация . . . 40
3.3.2 Вход и токены . . . 40
3.3.3 Основные API Auth Service . . . 41
3.4 User Service . . . 41
3.4.1 Синхронизация профилей . . . 41
3.4.2 Поиск пользователей . . . 41
3.4.3 Заявки и связи . . . 41
3.4.4 Тренировочные группы . . . 42
3.4.5 Внутренние API . . . 42
3.5 Training Service . . . 43
3.5.1 Тренировочные планы . . . 43
3.5.2 Назначения . . . 43
3.5.3 Просроченность задания . . . 44
3.5.4 Отчёты . . . 44

3.5.5 Шаблоны тренировок . . . 45
3.5.6 События Training Service . . . 45
3.6 Notification Service . . . 45
3.6.1 Обработка событий . . . 45
3.6.2 API уведомлений . . . 46
3.7 Analytics Service . . . 46
3.7.1 Сводка спортсмена . . . 46
3.7.2 Прогресс по периодам . . . 47
3.7.3 Обзор тренера . . . 47
3.8 AI Service . . . 47
3.8.1 Проверка доступа . . . 47
3.8.2 Получение данных . . . 48
3.8.3 Формирование prompt . . . 48
3.8.4 Генерация через Ollama . . . 48
3.9 Примеры API-запросов . . . 49
3.9.1 Регистрация пользователя . . . 49
3.9.2 Создание тренировочного плана . . . 49
3.9.3 Отправка отчёта . . . 49
3.9.4 Получение AI-рекомендации . . . 50
3.10 Тестирование . . . 50
3.10.1 Уровни тестирования . . . 50
3.10.2 Unit-тесты . . . 50
3.10.3 Компиляция интеграционных тестов . . . 51
3.10.4 E2E и интеграционные тесты . . . 51
3.10.5 Таблица результатов . . . 51
3.11 Выводы по главе 3 . . . 52
ЗАКЛЮЧЕНИЕ . . . 53
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ . . . 60
ПРИЛОЖЕНИЕ А Состав Backend-Сервисов . . . 61
ПРИЛОЖЕНИЕ Б Группы Публичного API . . . 62
ПРИЛОЖЕНИЕ В Примеры JSON-Запросов И Ответов . . . 63
ПРИЛОЖЕНИЕ Г ER-Диаграмма Основных Таблиц . . . 65
ПРИЛОЖЕНИЕ Д Sequence Diagrams Для Ключевых Сценариев . . . 68
ПРИЛОЖЕНИЕ Е События NATS JetStream . . . 71
ПРИЛОЖЕНИЕ Ж Таблица Тестирования . . . 72

ПРИЛОЖЕНИЕ И Команды Быстрого Запуска 73

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе бакалавра применяют следующие термины с соответствующими определениями:

Backend-платформа — серверная часть программной системы, включающая бизнес-логику, API, хранение данных и межсервисное взаимодействие

Микросервис — независимый сервис с ограниченной зоной ответственности, взаимодействующий с другими сервисами через API или сообщения

Микросервисная архитектура — архитектурный подход, при котором система строится как набор независимых сервисов, развиваемых и разворачиваемых отдельно

Событийное взаимодействие — асинхронный способ обмена, при котором сервис публикует событие, а подписчики обрабатывают его независимо

Stateless-сервис — сервис, не владеющий собственной предметной базой данных и вычисляющий результат по входным данным и данным других сервисов

Публичный API — внешний программный интерфейс платформы, доступный клиентским приложениям через API Gateway

Внутренний API — служебный программный интерфейс межсервисного взаимодействия, недоступный внешним клиентам

API Gateway — единая точка входа в backend-платформу, выполняющая проверку доступа и маршрутизацию запросов к внутренним сервисам

Связь тренер-спортсмен — подтвержденное отношение между тренером и спортсменом, определяющее права доступа к тренировочным данным

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе бакалавра применяют следующие сокращения и обозначения:

API — Application Programming Interface, программный интерфейс приложения

AI — Artificial Intelligence, искусственный интеллект

REST — Representational State Transfer, архитектурный стиль построения HTTP API

HTTP — HyperText Transfer Protocol, протокол передачи гипертекста

JWT — JSON Web Token, формат токена для передачи утверждений о пользователе

SQL — Structured Query Language, язык структурированных запросов к реляционным базам данных

DTO — Data Transfer Object, объект передачи данных между слоями или сервисами

UUID — Universally Unique Identifier, универсальный уникальный идентификатор

JSON — JavaScript Object Notation, текстовый формат обмена структурированными данными

JSONB — Binary JSON, бинарный тип хранения JSON-документов в PostgreSQL

CORS — Cross-Origin Resource Sharing, механизм контроля междоменных HTTP-запросов

NATS — система обмена сообщениями для событийного взаимодействия микросервисов

FCM — Firebase Cloud Messaging, сервис push-уведомлений

LLM — Large Language Model, большая языковая модель

E2E — End-to-End, сквозное тестирование пользовательского сценария

CI/CD — Continuous Integration/Continuous Delivery, непрерывная интеграция и доставка

БД — база данных

ВВЕДЕНИЕ

Современный тренировочный процесс всё чаще опирается не только на очное взаимодействие тренера и спортсмена, но и на постоянный обмен цифровыми данными. Тренер должен заранее формировать тренировочные задания, распределять их между спортсменами, учитывать индивидуальные особенности подготовки, получать обратную связь после выполнения тренировки и принимать решения о дальнейшем изменении нагрузки. При работе с одним спортсменом такие действия могут выполняться вручную и не требуют сложной информационной системы. Однако при увеличении количества спортсменов и появлении тренировочных групп ручной обмен сообщениями быстро становится неудобным и ненадёжным.

На практике взаимодействие тренера со спортсменами часто организуется с помощью мессенджеров, электронных таблиц и произвольных файлов. Тренер может отправлять задания в личные сообщения, групповые чаты или таблицы, а спортсмены присылают отчёты в удобном для них виде. Такой подход не требует внедрения специализированного программного обеспечения, но создаёт несколько существенных проблем.

Во-первых, отсутствует единый формат данных. Отчёты спортсменов отличаются по структуре, полноте и способу представления. Из-за этого тренеру приходится вручную приводить информацию к сопоставимому виду. Во-вторых, затрудняется поиск и анализ истории тренировок. Данные распределены по разным чатам, таблицам и файлам, поэтому их сложно агрегировать и использовать для оценки динамики подготовки. В-третьих, сообщения о тренировках смешиваются с личной и организационной перепиской. Тренер может пропустить важный отчёт или несвоевременно отреагировать на жалобу спортсмена. В-четвёртых, при работе с группами становится сложно контролировать, кому было назначено задание, кто его выполнил, кто отправил отчёт, а кто ещё нет.

Эти проблемы особенно заметны в циклических видах спорта, в которых важно не только получить отдельный отчёт, но и накопить историю структурированных данных. На основе такой истории можно рассчитывать сводные показатели, отслеживать прогресс, замечать признаки чрезмерной нагрузки и принимать более обоснованные решения при планировании следующих тренировок.

В связи с этим актуальной является задача разработки единой backend-платформы, которая автоматизирует взаимодействие тренера и спортсмена, стандартизирует формат тренировочных отчётов, обеспечивает хранение истории тренировок и предоставляет программные интерфейсы для внешних клиентских приложений. Такая платформа должна решать не только задачу обмена сообщениями, но и задачу организации предметной логики тренировочного процесса: ролей пользователей, связей между тренером и спортсменами, групп, планов, назначений, отчётов, уведомлений и аналитики.

В данной работе рассматривается backend-часть платформы CoachLink. Backend реализует серверную логику, хранение данных, публичные REST API, межсервисное взаимодействие и интеграцию с инфраструктурными компонентами. Мобильное приложение рассматривается только как внешний клиент, который обращается к backend через API Gateway. Разработка мобильного приложения не входит в область ответственности данной выпускной квалификационной работы.

Такое разграничение важно, поскольку основная инженерная задача данной работы заключается не в проектировании пользовательского интерфейса, а в создании устойчивой серверной основы для платформы. Именно backend определяет модель данных, правила доступа, жизненный цикл тренировочных заданий, структуру отчётов, механизм уведомлений и возможность дальнейшей аналитики.

Платформа CoachLink проектируется как расширяемое средство для автоматизации тренировочного процесса в разных видах спорта. Первая реализация ориентирована только на лёгкую атлетику — это сделано для упрощения разработки первого прототипа. При этом архитектура платформы остаётся универсальной: базовые сущности (пользователь, тренер, спортсмен, группа, план, назначение, отчёт, уведомление) не привязаны к конкретному виду спорта и в будущем могут быть дополнены специфическими метриками и шаблонами для других дисциплин.

Объектом исследования является процесс информационного взаимодействия между тренером и спортсменом при планировании, выполнении и анализе тренировок.

Предметом исследования является backend-платформа для автоматизации назначения тренировочных заданий, сбора стандартизированных отчётов, отправки уведомлений и анализа данных

тренировочного процесса.

Целью выпускной квалификационной работы является разработка открытой микросервисной backend-платформы CoachLink для стандартизации и автоматизации взаимодействия в системе «тренер-спортсмен».

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ существующих способов организации взаимодействия тренера и спортсмена.
2. Выявить ограничения мессенджеров, электронных таблиц, спортивных трекеров и закрытых платформ при решении задачи автоматизации тренировочного процесса.
3. Сформулировать функциональные и нефункциональные требования к backend-платформе.
4. Спроектировать микросервисную архитектуру системы и определить ответственность каждого сервиса.
5. Спроектировать модель данных для пользователей, связей, групп, тренировочных планов, назначений, отчётов и уведомлений.
6. Реализовать сервис авторизации с поддержкой JWT access token и refresh token.
7. Реализовать сервис пользователей, отвечающий за профили, связи тренер-спортсмен и тренировочные группы.
8. Реализовать сервис тренировок, отвечающий за планы, назначения, шаблоны и отчёты.
9. Реализовать сервис уведомлений, создающий уведомления на основе событий системы.
10. Реализовать сервис аналитики, агрегирующий данные тренировочного процесса.
11. Реализовать AI-сервис рекомендаций по последним отчётам спортсмена.
12. Описать публичные API с помощью OpenAPI.
13. Провести тестирование backend-компонентов и зафиксировать подтверждённые результаты.

При выполнении работы используются методы системного анализа, объектно-ориентированного и компонентного проектирования, проектирования REST API, проектирования реляционных баз данных и

тестирования программного обеспечения. Выбор микросервисной архитектуры соотносится с подходами к построению распределённых систем, описанными в работах по шаблонам микросервисов, проектированию независимых сервисов и переходу к cloud-native архитектурам [1; 2; 3; 4].

В качестве основного языка реализации backend выбран Go [5]. Для разработки HTTP API используется Echo [6]. Для хранения данных применяется PostgreSQL [7]. Для работы с SQL-запросами используется `sqlx`, для миграций — `goose`. Для аутентификации применяется JWT [8], для хеширования паролей — bcrypt. Асинхронное межсервисное взаимодействие реализовано через NATS JetStream [9]. Для локального развёртывания используется Docker Compose [10]. API документируется в формате OpenAPI [11]. AI-рекомендации формируются через локальный LLM-сервер Ollama [12].

Научная значимость работы заключается в проектировании модели backend-платформы, объединяющей микросервисный подход, событийное взаимодействие, стандартизированный сбор тренировочных данных и аналитическую обработку этих данных. Работа демонстрирует, как предметная область тренировочного процесса может быть представлена через набор независимых сервисов с чёткими зонами ответственности.

Практическая значимость заключается в создании открытой серверной основы для платформы CoachLink. Реализованный backend может использоваться как база для дальнейшего развития системы: подключения мобильного приложения, добавления новых видов спорта, расширения аналитики, интеграции со спортивными устройствами и развития AI-сценариев. Благодаря открытому API систему можно адаптировать под разные клиентские приложения и внешние интеграции.

Выпускная квалификационная работа состоит из введения, трёх глав, заключения и приложений.

В первой главе проводится аналитический обзор существующих подходов к организации взаимодействия тренера и спортсмена. Рассматриваются мессенджеры, электронные таблицы, спортивные трекеры и закрытые платформы. На основе сравнения формулируются требования к backend-платформе.

Во второй главе описывается проектирование CoachLink. Рассматриваются архитектура микросервисов, модель данных, межсервисное

взаимодействие, ключевые пользовательские сценарии и выбор технологического стека.

В третьей главе описывается практическая реализация backend-сервисов и результаты тестирования. Отдельно рассматриваются API Gateway, Auth Service, User Service, Training Service, Notification Service, Analytics Service и AI Service.

В заключении подводятся итоги работы, оценивается достижение цели и формулируются направления дальнейшего развития платформы.

1 Аналитический обзор

1.1 Особенности взаимодействия тренера и спортсмена

Тренировочный процесс представляет собой повторяющийся цикл планирования, выполнения, отчётности и корректировки нагрузки. Тренер формирует задание, спортсмен выполняет его и передаёт обратную связь, после чего тренер оценивает состояние спортсмена и принимает решение о следующем этапе подготовки. Для эффективной работы важно, чтобы этот цикл был регулярным, прозрачным и основанным на достоверных данных.

В классической форме взаимодействие тренера и спортсмена может происходить очно: тренер объясняет задание на тренировке, наблюдает за выполнением и сразу корректирует действия спортсмена. Однако в реальных условиях такая модель не всегда достаточна. Спортсмены могут выполнять часть тренировок самостоятельно, находиться в разных местах, тренироваться в разное время или входить в разные группы. Поэтому тренеру необходим цифровой инструмент, позволяющий сопровождать тренировочный процесс вне очных занятий.

Цифровая поддержка тренировочного процесса должна решать несколько задач:

1. Передача тренировочных заданий спортсменам.
2. Разделение индивидуальных и групповых назначений.
3. Получение отчётов о выполнении заданий.
4. Хранение истории тренировок.
5. Уведомление участников о важных событиях.
6. Анализ накопленных данных.
7. Поддержка разных ролей и прав доступа.

Если рассматривать систему только как средство отправки сообщений, можно ограничиться мессенджером. Но при необходимости хранить историю тренировок, связывать отчёт с конкретным заданием и выполнять аналитику простого обмена сообщениями становится недостаточно. Требуется система, в которой данные имеют заранее определённую структуру.

1.2 Использование мессенджеров

Мессенджеры являются самым распространённым способом цифрового

взаимодействия, потому что они уже установлены у большинства пользователей и не требуют дополнительного обучения. Тренер может создать общий чат группы, отправить задание или написать спортсмену лично. Спортсмен, в свою очередь, может быстро ответить текстом, голосовым сообщением, фотографией или файлом.

Главное преимущество мессенджеров — низкий порог входа. Для начала работы не требуется разрабатывать отдельную систему, создавать аккаунты в специализированном сервисе или обучать пользователей. Мессенджер удобен для оперативного общения и решения организационных вопросов.

Однако при использовании мессенджеров для ведения тренировочного процесса возникают существенные ограничения.

Во-первых, мессенджер не задаёт структуру тренировочного отчёта. Спортсмен сам решает, что написать и в каком формате. Один спортсмен может указать дистанцию и пульс, другой — только общее самочувствие, третий — прислать фотографию часов или блокнота. В результате тренер получает разноформатные данные.

Во-вторых, отчёты трудно связывать с конкретными заданиями. Если тренер назначил несколько тренировок, а спортсмен прислал сообщение спустя несколько дней, необходимо вручную понимать, к какому заданию оно относится. Это особенно неудобно при групповой работе.

В-третьих, история тренировок оказывается смешана с другой перепиской. В чате могут быть организационные сообщения, личные вопросы, обсуждения соревнований и отчёты. Поиск нужной информации становится затруднительным.

В-четвёртых, мессенджеры не предоставляют специализированной аналитики. Даже если все отчёты сохранены в переписке, рассчитать процент выполнения заданий, динамику нагрузки или суммарную дистанцию без ручной обработки невозможно.

Следовательно, мессенджер удобен как канал общения, но не является полноценной платформой для управления тренировочным процессом.

1.3 Использование электронных таблиц

Электронные таблицы частично решают проблему структуры данных. Тренер может создать таблицу с колонками для даты, задания, длительности, дистанции, пульса и комментария. Спортсмены могут заполнять строки после

выполнения тренировок, а тренер — просматривать данные и выполнять простые расчёты.

Преимущество таблиц заключается в том, что они позволяют хранить историю тренировок в более структурированном виде, чем мессенджеры. Таблицы также поддерживают фильтрацию, сортировку, формулы и визуализацию. Для небольших групп этого может быть достаточно.

Однако у таблиц также есть ограничения.

Во-первых, таблицы не обеспечивают полноценную ролевую модель. Настроить доступ можно на уровне всего документа или листа, но сложно выразить предметные правила: спортсмен видит только свои задания, тренер видит всех своих спортсменов, участник группы видит только группы, в которых состоит.

Во-вторых, таблицы плохо подходят для сценария назначения тренировок. Они хранят данные, но не моделируют жизненный цикл задания: создано, назначено, выполнено, заархивировано. Неочевидно, кто должен заполнить конкретную строку и было ли задание действительно выполнено.

В-третьих, отсутствует встроенный механизм уведомлений. Тренер не получает отдельное системное уведомление о том, что спортсмен отправил отчёт. Спортсмен также может не увидеть новое задание без дополнительного сообщения в мессенджере.

В-четвёртых, при большом количестве спортсменов таблица становится сложной для поддержки. Появляется риск случайного изменения чужих данных, удаления строк, нарушения формул и несогласованности формата.

Таким образом, электронные таблицы лучше мессенджеров подходят для хранения истории, но не обеспечивают полноценную автоматизацию взаимодействия.

1.4 Спортивные трекеры и приложения

Спортивные трекеры и приложения предназначены для фиксации фактических показателей активности. Они могут автоматически собирать дистанцию, темп, пульс, маршрут, высоту, каденс и другие параметры. Для лёгкой атлетики такие инструменты полезны, потому что позволяют получить объективные данные о тренировке.

Сильной стороной спортивных трекеров является автоматизация измерений. Спортсмену не нужно вручную вводить все показатели, если

устройство уже записало тренировку. Также многие приложения предоставляют графики и базовую аналитику.

Однако спортивные трекеры не полностью решают задачу взаимодействия тренера и спортсмена.

Во-первых, они обычно ориентированы на факт выполненной активности, а не на управление назначениями. Важная для тренера логика «задание было назначено, спортсмен его выполнил, отчёт связан с назначением» может отсутствовать или быть ограниченной.

Во-вторых, трекеры не всегда учитывают контекст тренера: тренировочные группы, индивидуальные планы, заявки на связь, ролевой доступ, уведомления о новых отчётах.

Во-третьих, такие сервисы часто являются закрытыми платформами. Их API может быть ограничен, платным или недоступным для самостоятельного расширения. Это снижает гибкость при разработке открытой платформы.

В-четвёртых, не все тренировки удобно фиксировать автоматически. Спортсмен может выполнять силовую подготовку, специальные беговые упражнения, восстановительную работу или техническую тренировку. В таких случаях текстовый комментарий и субъективная оценка нагрузки остаются важными.

Следовательно, спортивные трекеры полезны как потенциальный источник данных, но не заменяют backend-платформу взаимодействия тренера и спортсмена.

1.5 Закрытые коммерческие платформы

На рынке существуют специализированные платформы для тренеров, которые позволяют назначать тренировки, отслеживать выполнение, работать с группами и анализировать данные. Такие решения ближе всего к требованиям предметной области.

Их преимуществами являются готовый пользовательский интерфейс, наличие функциональности для тренеров, интеграции с устройствами и развитая аналитика. Однако для дипломного проекта и открытой платформы у закрытых решений есть ряд недостатков.

Первый недостаток — ограниченная расширяемость. Если платформа не предоставляет открытого API или возможности самостоятельной доработки, её сложно адаптировать под нестандартные требования.

Второй недостаток — зависимость от внешнего поставщика. Пользователь не контролирует архитектуру, хранение данных, правила доступа и дальнейшее развитие продукта.

Третий недостаток — стоимость. Многие профессиональные платформы ориентированы на коммерческое использование и могут быть недоступны для небольших групп, начинающих тренеров или учебных проектов.

Четвёртый недостаток — невозможность использовать такую платформу как основу для исследовательской и инженерной работы. В рамках выпускной квалификационной работы важно не только применить готовое решение, но и спроектировать и реализовать собственную архитектуру.

Таким образом, закрытые платформы подтверждают востребованность задачи, но не заменяют разработку открытой backend-системы.

1.6 Сравнительный анализ подходов

Для выбора направления разработки сравним рассмотренные подходы по критериям, важным для автоматизации тренировочного процесса.

Таблица 1 – Сравнение подходов к организации взаимодействия тренера и спортсмена

Критерий	Мессендж.	Эл. табл.	Трекеры	Закрытые платформы	CoachLink
Единый формат отчётов	Нет	Частично	Да, для активности	Да	Да
Связь отчёта с заданием	Нет	Частично	Ограниченно	Да	Да
Групповые назначения	Частично	Частично	Ограниченно	Да	Да
Роли тренера и спортсмена	Нет	Ограниченно	Частично	Да	Да
Уведомления о событиях	Да, но не профильные	Нет	Да	Да	Да
Хранение истории	Да, в разном формате	Да	Да	Да	Да
Аналитика	Нет	Базовая	Да, по активности	Да	Да

Таблица 1 – Сравнение подходов к организации взаимодействия тренера и спортсмена

Критерий	Мессендж.	Эл. табл.	Трекеры	Закрытые платформы	CoachLink
Открытость API	Нет	Ограниченно	Зависит от сервиса	Зависит от сервиса	Да
Возможность доработки	Низкая	Средняя	Низкая	Низк./сред.	Высокая
Самостоятельное развёртывание	Нет	Частично	Нет	Обычно нет	Да

Из таблицы видно, что ни один из распространённых подходов полностью не удовлетворяет требованиям открытой backend-платформы. Мессенджеры и таблицы удобны как вспомогательные инструменты, но не обеспечивают предметную модель. Спортивные трекеры полезны как источник данных, но не покрывают весь процесс взаимодействия. Закрытые платформы функциональны, но ограничены с точки зрения открытости и самостоятельного развития.

CoachLink ориентируется на сочетание нескольких свойств: открытость, структурированность данных, ролевой доступ, групповые назначения, уведомления, аналитика и расширяемость. Это делает разработку собственной backend-платформы обоснованной.

1.7 Требования к разрабатываемой backend-платформе

На основе анализа можно сформулировать функциональные требования.

1.7.1 Пользователи и роли

Система должна поддерживать регистрацию пользователей и две базовые роли: тренер и спортсмен. Роль должна использоваться при проверке доступа к функциям. Например, только тренер может создавать тренировочные планы и группы, а отправлять отчёт по назначению может только спортсмен, которому это назначение принадлежит.

1.7.2 Связь тренер-спортсмен

Система должна поддерживать механизм установления связи между

спортсменом и тренером. Для этого спортсмен отправляет заявку, а тренер принимает или отклоняет её. Такой подход защищает тренера от автоматического добавления спортсменов и позволяет явно контролировать состав своей группы.

1.7.3 Тренировочные группы

Тренер должен иметь возможность создавать группы спортсменов. Группа нужна для массового назначения тренировок нескольким спортсменам сразу. При этом добавлять в группу можно только тех спортсменов, которые уже связаны с данным тренером.

1.7.4 Тренировочные планы и назначения

Система должна позволять тренеру создавать тренировочные планы, назначать их отдельным спортсменам или группе и просматривать список назначений. Назначение должно иметь статус, позволяющий отличать активные, выполненные и архивные задания.

1.7.5 Отчёты

Спортсмен должен иметь возможность отправить отчёт по назначенному заданию. Отчёт должен иметь единый формат. Для первой версии, ориентированной на лёгкую атлетику, в отчёт включаются текстовый комментарий, длительность, субъективная нагрузка, дистанция и показатели пульса.

1.7.6 Уведомления

Система должна создавать уведомления о важных событиях: заявке на связь, принятии заявки, назначении тренировки, удалении задания, отправке отчёта, добавлении или удалении из группы. Уведомления должны храниться в базе данных и быть доступны через API.

1.7.7 Аналитика

Система должна рассчитывать сводные показатели по тренировочным данным. Минимальный набор аналитики включает количество отчётов, суммарную длительность, дистанцию, среднюю нагрузку, пульс и процент

выполнения назначений.

1.7.8 AI-рекомендации

Система должна предоставлять тренеру возможность получить текстовую рекомендацию по конкретному спортсмену на основе последних отчётов и сводной статистики. AI-модуль должен быть вспомогательным инструментом и не заменять решение тренера.

1.8 Нефункциональные требования

К нефункциональным требованиям относятся:

1. Микросервисная архитектура, позволяющая независимо развивать компоненты.
2. REST API как основной способ взаимодействия внешних клиентов с backend.
3. OpenAPI-спецификация для документирования контрактов.
4. Разделение данных по сервисам.
5. Использование PostgreSQL для надёжного хранения данных.
6. Использование брокера сообщений для асинхронных операций.
7. Контейнеризация для локального развёртывания.
8. Возможность расширения предметной модели под другие виды спорта.
9. Тестируемость бизнес-логики.
10. Минимальная зависимость от конкретного клиентского приложения.

1.9 Выводы по главе 1

В первой главе были рассмотрены основные способы организации цифрового взаимодействия тренера и спортсмена: мессенджеры, электронные таблицы, спортивные трекеры и закрытые платформы. Анализ показал, что каждый из подходов решает только часть задачи.

Мессенджеры удобны для общения, но не обеспечивают структурированное хранение данных. Электронные таблицы позволяют хранить историю, но не обладают полноценной предметной логикой, уведомлениями и ролевой моделью. Спортивные трекеры собирают данные активности, но не организуют полный цикл назначения и отчётности.

Закрытые платформы функциональны, но ограничены с точки зрения открытости, самостоятельного развёртывания и доработки.

На основании анализа сформулированы требования к backend-платформе CoachLink. Система должна поддерживать роли пользователей, связи тренера и спортсмена, тренировочные группы, планы, назначения, стандартизированные отчёты, уведомления, аналитику и AI-рекомендации. Для удовлетворения этих требований в следующей главе предлагается микросервисная архитектура backend-платформы.

2 Проектная часть

2.1 Общий подход к проектированию

Целью проектной части является построение backend-архитектуры, которая покрывает основные сценарии взаимодействия тренера и спортсмена и при этом остаётся расширяемой. Система должна быть пригодна для развития: добавления новых видов спорта, подключения внешних клиентов, появления дополнительных аналитических модулей и интеграций со спортивными устройствами.

Для решения этой задачи выбран микросервисный подход. Он позволяет разделить систему на независимые компоненты по предметным областям. В CoachLink такими областями являются авторизация, профили и связи пользователей, тренировочный процесс, уведомления, аналитика и AI-рекомендации.

Монолитная архитектура для первой версии могла бы быть проще в реализации, однако она хуже демонстрирует независимость предметных модулей и сложнее масштабируется организационно. В системе «тренер-спортсмен» отдельные части имеют разную природу. Например, сервис авторизации отвечает за безопасность и токены, сервис тренировок — за предметную логику назначений и отчётов, сервис уведомлений — за реакцию на события, а сервис аналитики — за расчёт показателей. Разделение этих частей повышает ясность архитектуры.

Проектирование выполнялось с учётом следующих принципов:

1. Единая точка входа для внешних клиентов.
2. Разделение ответственности между сервисами.
3. Собственная база данных для stateful-сервисов.
4. Использование событий для асинхронных операций.
5. Использование внутренних API для синхронных межсервисных запросов.
6. Документирование публичного API.
7. Возможность расширения предметной модели.

2.2 Высокоуровневая архитектура

Внешний клиент взаимодействует с платформой через API Gateway.

Клиентом может быть мобильное приложение, web-интерфейс или другой сервис. В рамках данной работы клиентская часть не рассматривается как результат разработки, а используется только как потребитель REST API.

API Gateway проверяет авторизацию и перенаправляет запросы к backend-сервисам. Сервисы реализованы на Go и взаимодействуют между собой через HTTP и NATS JetStream. Для хранения данных используется PostgreSQL. Для генерации AI-рекомендаций используется локальный LLM-сервер Ollama.

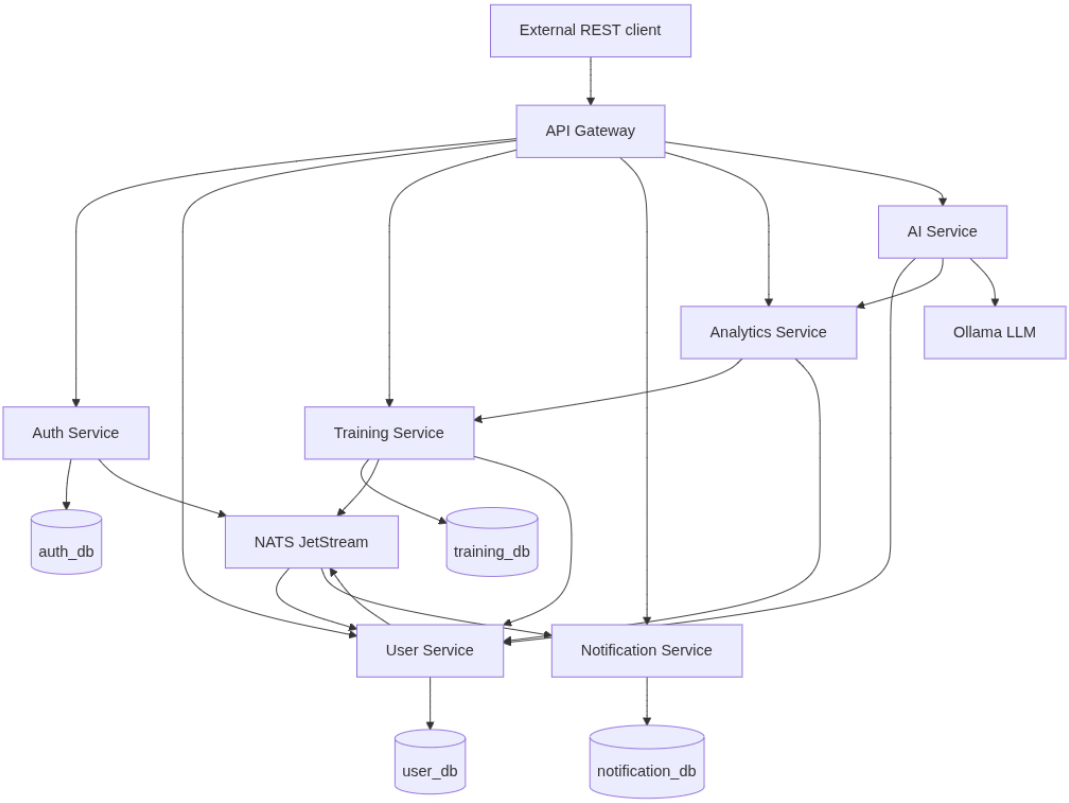

Рисунок 1 – Компонентная схема backend-платформы

В такой архитектуре API Gateway не содержит предметной бизнес-логики. Его задача — принять запрос, проверить JWT, добавить служебные заголовки и передать запрос в нужный сервис. Предметные правила остаются внутри соответствующих сервисов.

2.3 Состав сервисов

2.3.1 API Gateway

API Gateway является входной точкой для всех публичных HTTP-запросов. Он решает следующие задачи:

– маршрутизация запросов по префиксам;
– проверка JWT access token;
– извлечение идентификатора пользователя, роли и логина из токена;
– проброс пользовательского контекста во внутренние сервисы через заголовки;
– обработка CORS;
– предоставление health-check endpoint.

Публичные маршруты без авторизации ограничены регистрацией, входом и обновлением токена:

– `POST /api/v1/auth/register`;
– `POST /api/v1/auth/login`;
– `POST /api/v1/auth/refresh`.

Остальные маршруты требуют заголовок `Authorization: Bearer <token>`. После успешной проверки Gateway устанавливает заголовки:

– `X-User-ID`;
– `X-User-Role`;
– `X-User-Login`.

Такой подход позволяет не проверять JWT повторно в каждом сервисе. Внутренние сервисы доверяют Gateway как единой точке входа и используют переданные заголовки для проверки прав доступа.

2.3.2 Auth Service

Auth Service отвечает за учётные данные и токены. В его область ответственности входят:

– регистрация пользователя;
– валидация логина, email, пароля и роли;
– хеширование пароля;
– вход пользователя;
– выпуск access token и refresh token;
– обновление пары токенов;
– отзыв refresh token при logout;
– публикация события регистрации.

Access token реализован как JWT. В payload токена включаются:

```
1 {
```

```
2   "sub": "uuid пользователя",
3   "login": "user_login",
4   "role": "coach | athlete",
5   "exp": 1234567890,
6   "iat": 1234567890
7 }
```

Refresh token хранится в базе данных не в открытом виде, а в виде SHA-256 хеша. Это снижает риск компрометации токена при утечке базы данных. При refresh старый токен удаляется, а пользователю выдаётся новая пара токенов.

После успешной регистрации Auth Service публикует событие `coachlink.user.registered`. Это событие используется User Service для создания пользовательского профиля.

2.3.3 User Service

User Service отвечает за доменную информацию о пользователях. Он не хранит пароли и не выпускает токены. Его задачи:

- хранение профилей пользователей;
- синхронизация профилей из события регистрации;
- поиск пользователей по ФИО и логину;
- управление заявками спортсмена тренеру;
- управление связями тренер-спортсмен;
- управление тренировочными группами;
- предоставление внутренних API для других сервисов.

Связь тренера и спортсмена создаётся через заявку. Это решение выбрано для явного контроля доступа: спортсмен не может автоматически прикрепиться к тренеру без подтверждения. Тренер видит входящие заявки и может принять или отклонить их.

Тренировочные группы принадлежат тренеру. Добавлять в группу можно только спортсменов, которые уже связаны с этим тренером. Это предотвращает ситуацию, когда тренер назначает тренировку спортсмену, не входящему в его область ответственности.

2.3.4 Training Service

Training Service реализует центральную предметную логику платформы.

Он отвечает за:

- создание тренировочных планов;
- назначение планов спортсменам;
- назначение планов группе;
- получение списка назначений;
- фильтрацию назначений;
- удаление и архивирование назначений;
- отправку отчётов;
- хранение шаблонов тренировок;
- предоставление внутренних API для аналитики.

Тренировочный план содержит название, описание и дату. Назначение связывает план с конкретным спортсменом. Такое разделение позволяет одному плану соответствовать нескольким назначениям, если тренировка выдана группе.

Назначение имеет статус:

- `assigned` — задание назначено и ожидает отчёта;
- `completed` — спортсмен отправил отчёт;
- `archived` — тренер перенёс выполненное задание в архив.

Просроченность не хранится как отдельное поле, а вычисляется при чтении. Это уменьшает риск рассинхронизации данных. Назначение считается просроченным, если оно находится в статусе `assigned`, а дата выполнения уже прошла.

2.3.5 Notification Service

Notification Service создаёт и хранит уведомления. Он подписывается на события NATS JetStream и реагирует на них созданием записи в таблице `notifications`.

Сервис обрабатывает события:

- заявка спортсмена тренеру;
- принятие или отклонение заявки;
- назначение тренировки;
- удаление задания;
- отправка отчёта;
- добавление спортсмена в группу;
- удаление спортсмена из группы.

Пользователь может получить список уведомлений, отфильтровать их по признаку прочтения, получить количество непрочитанных уведомлений в ответе списка и отметить уведомления как прочитанные. Также сервис хранит FCM-токены устройств. Если Firebase credentials не настроены, push-доставка отключается, но уведомления продолжают сохраняться в базе данных.

2.3.6 Analytics Service

Analytics Service является stateless-сервисом. Он не имеет собственной базы данных, а получает данные из Training Service через внутренние API. Такое решение упрощает первую версию системы и исключает проблему синхронизации аналитической базы с основной базой тренировок.

Сервис рассчитывает:

– сводку по спортсмену;
– прогресс по неделям или месяцам;
– обзор по спортсменам тренера.

Ключевые метрики:

– количество отчётов;
– суммарная длительность;
– средняя длительность;
– суммарная дистанция;
– средняя воспринимаемая нагрузка;
– средний пульс;
– максимальный пульс;
– количество назначений;
– процент выполнения.

2.3.7 AI Service

AI Service предоставляет одну публичную функцию: генерацию рекомендаций по тренировочному процессу конкретного спортсмена.

Публичный endpoint:

```
POST /api/v1/ai/athletes/{id}/recommendations
```

Сервис доступен только тренеру. Перед генерацией он проверяет через User Service, что спортсмен действительно связан с текущим тренером. Затем AI Service получает из Analytics Service сводную статистику и последние

отчёты спортсмена. Количество отчётов ограничено пятью, чтобы prompt оставался компактным. Типовой сценарий — рекомендации по последним 3-5 тренировкам.

Генерация выполняется через Ollama. Prompt составляется на русском языке и содержит конкретные данные: объём тренировок, длительность, дистанцию, субъективную нагрузку, пульс и комментарии спортсмена. AI-модуль является вспомогательным инструментом и не заменяет решение тренера.

2.4 Модель данных

2.4.1 Auth Service

Auth Service использует две основные таблицы:

Таблица 2 – Таблицы Auth Service и их назначение

Таблица	Назначение
`users`	Учётные данные пользователя, роль и пароль в виде bcrypt-хеша
`refresh_tokens`	Хеши refresh-токенов и срок их действия

Таблица `users` содержит логин, email, ФИО, роль, хеш пароля и временные метки. Роль ограничена значениями `coach` и `athlete`. Таблица `refresh_tokens` связана с пользователем и позволяет отзывать refresh-токены.

2.4.2 User Service

User Service использует следующие таблицы:

Таблица 3 – Таблицы User Service и их назначение

Таблица	Назначение
`user_profiles`	Профили пользователей, синхронизированные из Auth Service
`connection_requests`	Заявки спортсмена тренеру
`coach_athlete_relations`	Активные связи тренер-спортсмен

Таблица 3 – Таблицы User Service и их назначение

Таблица	Назначение
`training_groups`	Тренировочные группы тренера
`training_group_members`	Состав тренировочных групп

Разделение `users` и `user_profiles` отражает разделение ответственности сервисов. Auth Service хранит учётные данные, User Service — доменную информацию для поиска, связей и групп.

2.4.3 Training Service

Training Service использует таблицы:

Таблица 4 – Таблицы Training Service и их назначение

Таблица	Назначение
`training_plans`	Общая информация о тренировочном плане
`training_assignments`	Назначение плана конкретному спортсмену
`training_reports`	Отчёт спортсмена по назначению
`training_templates`	Шаблоны тренировок тренера

Отдельное хранение плана и назначения необходимо для групповых тренировок. Один план может быть создан тренером один раз, а затем назначен нескольким спортсменам. Отчёт связан с назначением, а не только с планом, потому что каждый спортсмен выполняет тренировку индивидуально и отправляет собственную обратную связь.

2.4.4 Notification Service

Notification Service хранит:

Таблица 5 – Таблицы Notification Service и их назначение

Таблица	Назначение
`notifications`	Уведомления пользователей
`device_tokens`	FCM-токены устройств

В уведомлении хранится тип, заголовок, текст, дополнительные данные в JSONB, признак прочтения и дата создания. JSONB-поле позволяет добавлять к уведомлению контекст: идентификатор заявки, задания или группы.

2.4.5 ER-диаграмма

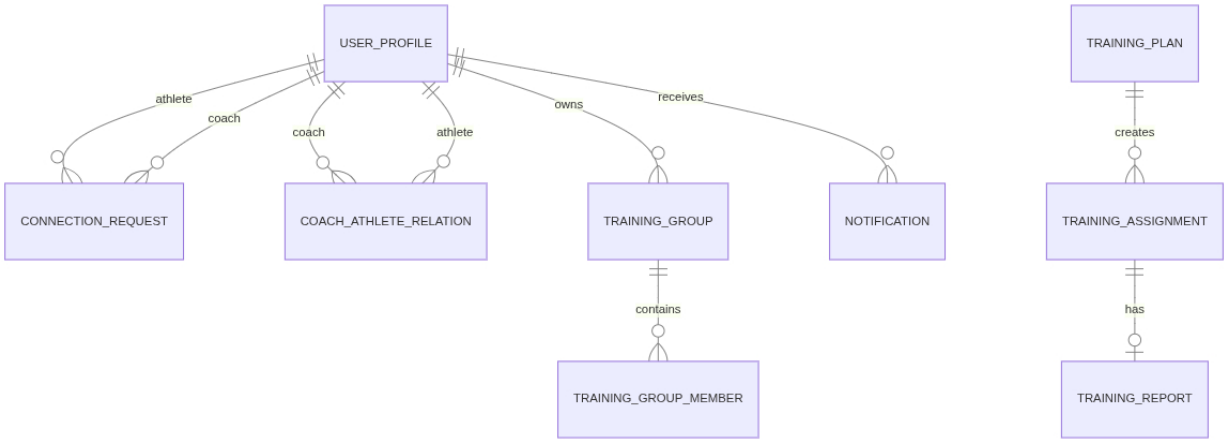

Рисунок 2 – ER-диаграмма

2.5 Межсервисное взаимодействие

2.5.1 Синхронное взаимодействие

Синхронные HTTP-вызовы используются в случаях, когда результат нужен для выполнения текущего запроса.

Примеры:

1. Training Service обращается к User Service при назначении плана группе, чтобы получить список участников группы.
2. Training Service получает профиль спортсмена, чтобы сохранить имя и логин в назначении.
3. Analytics Service обращается к Training Service за отчётами и агрегатами.
4. AI Service обращается к User Service для проверки связи тренера и спортсмена.
5. AI Service обращается к Analytics Service за сводкой и последними отчётами.

Внутренние API не предназначены для внешних клиентов. Они

используются только сервисами внутри backend-платформы.

2.5.2 Асинхронное взаимодействие

Асинхронное взаимодействие реализовано через NATS JetStream. Оно используется там, где действие может быть выполнено после основного запроса и не должно блокировать пользователя.

Например, при создании тренировочного назначения Training Service публикует событие `coachlink.training.assigned`. Notification Service получает событие и создаёт уведомление для спортсмена. Если Notification Service временно недоступен, JetStream durable consumer позволяет обработать событие после восстановления сервиса.

Список ключевых событий:

Таблица 6 – Ключевые события NATS JetStream в backend-платформе

Событие	Источник	Получатель	Назначение
`coachlink.user.registered`	Auth Service	User Service	Создание профиля
`coachlink.connection.requested`	User Service	Notification Service	Уведомление тренера
`coachlink.connection.accepted`	User Service	Notification Service	Уведомление спортсмена
`coachlink.connection.rejected`	User Service	Notification Service	Уведомление спортсмена
`coachlink.group.athlete_added`	User Service	Notification Service	Уведомление спортсмена
`coachlink.group.athlete_removed`	User Service	Notification Service	Уведомление спортсмена
`coachlink.training.assigned`	Training Service	Notification Service	Уведомление спортсмена
`coachlink.training.deleted`	Training Service	Notification Service	Уведомление спортсмена

Таблица 6 – Продолжение таблицы 6

Событие	Источник	Получатель	Назначение
`coachlink.report.submitted`	Training Service	Notification Service	Уведомление тренера

2.6 Проектирование ключевых сценариев

2.6.1 Регистрация пользователя

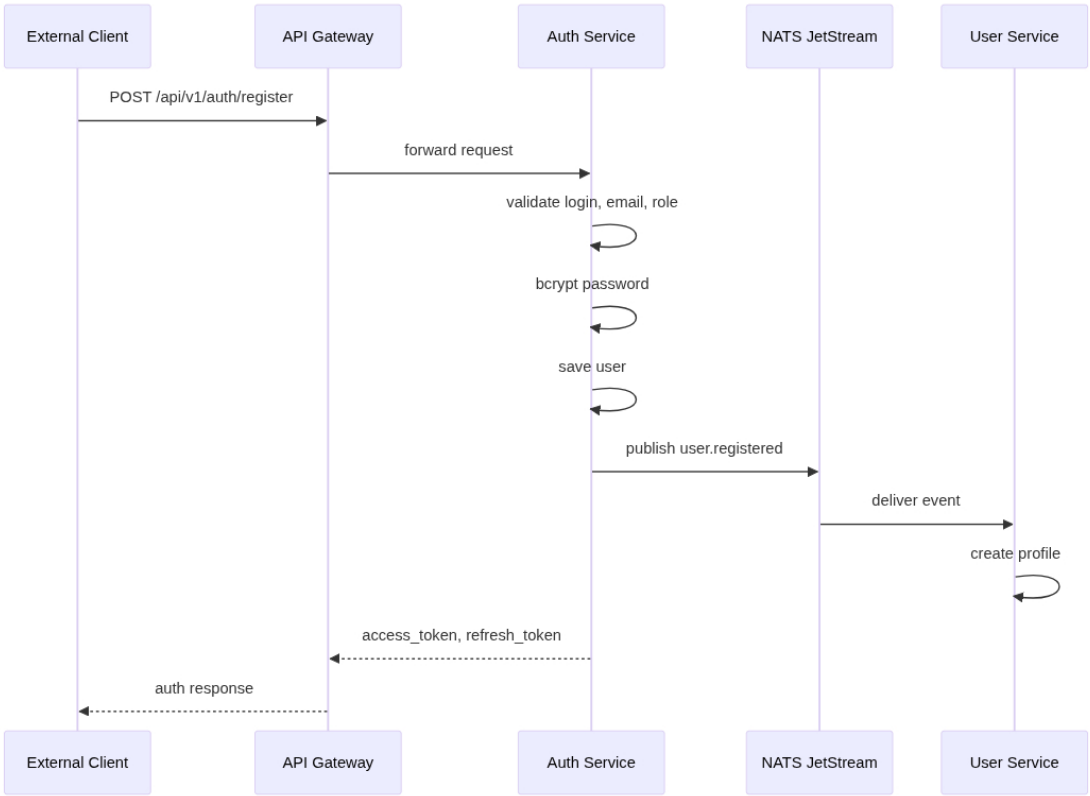

Рисунок 3 – Регистрация пользователя

2.6.2 Создание связи тренер-спортсмен

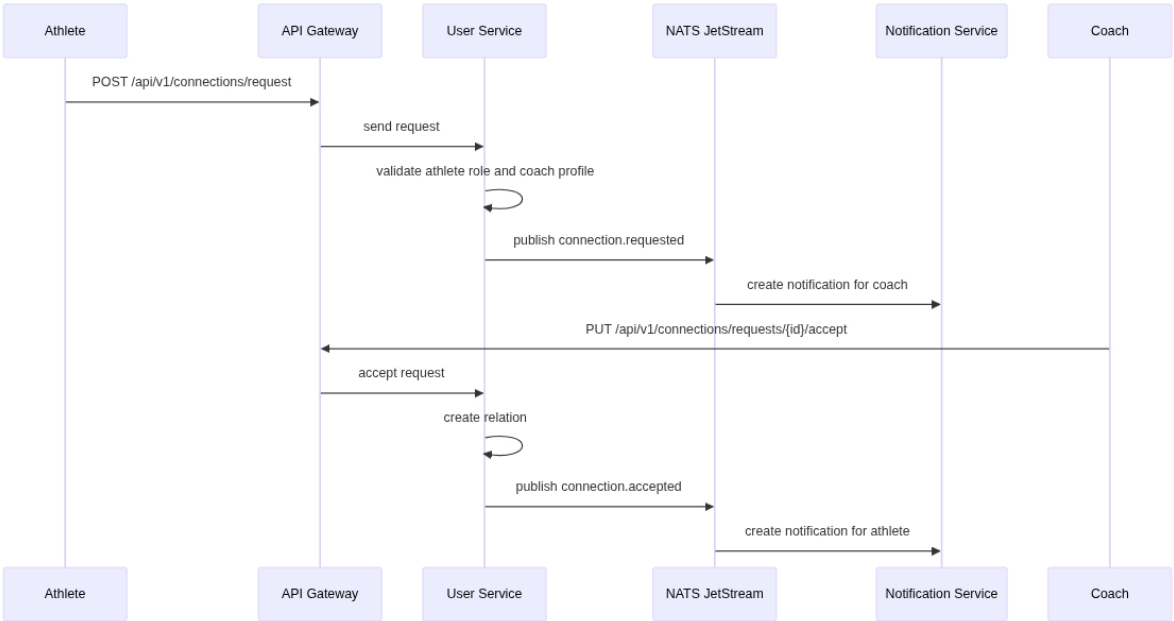

Рисунок 4 – Создание связи тренер-спортсмен

2.6.3 Назначение тренировки группе

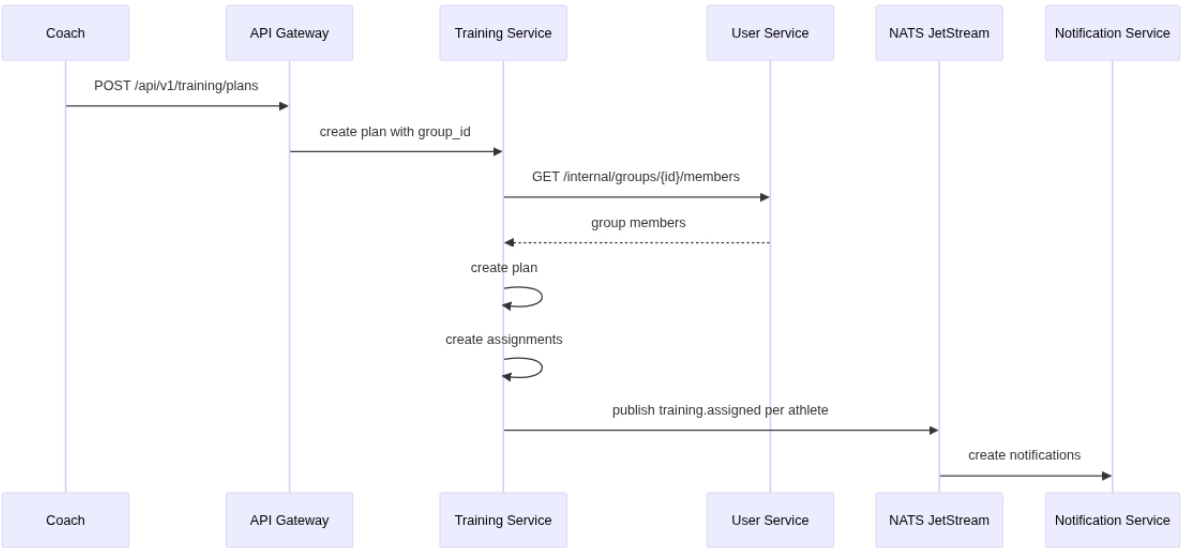

Рисунок 5 – Назначение тренировки групп

2.6.4 Отправка отчёта

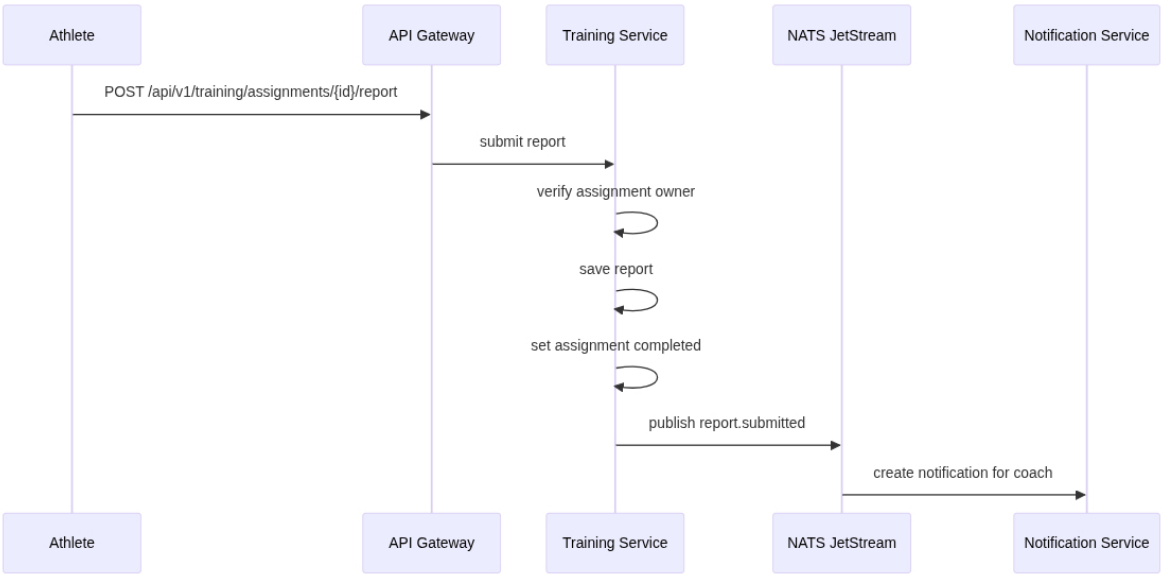

Рисунок 6 – Отправка отчёта

2.6.5 Получение AI-рекомендации

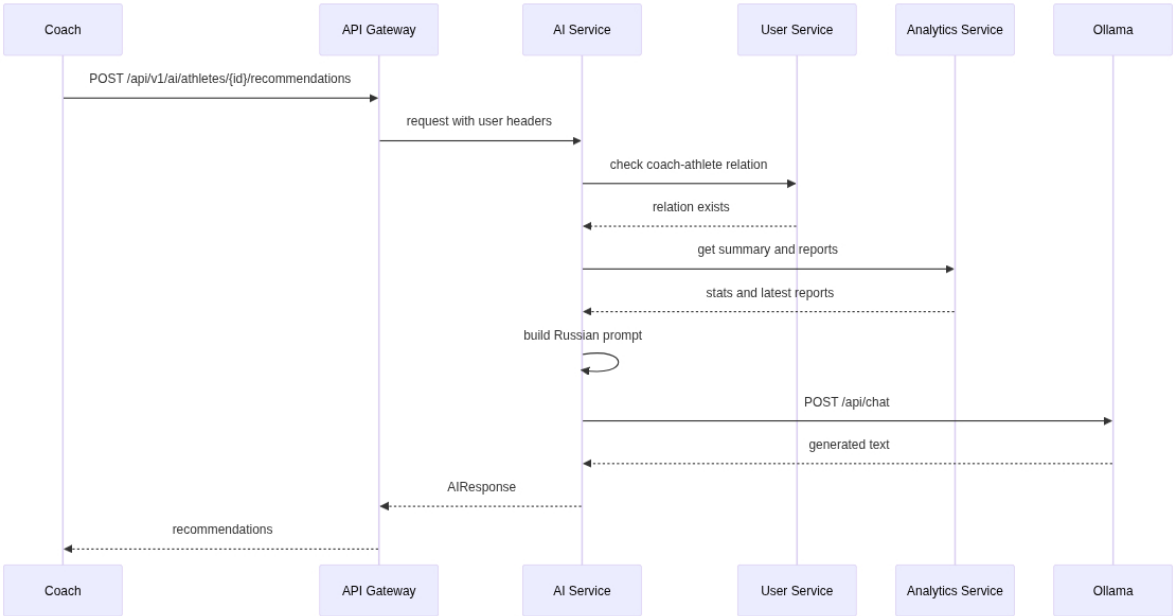

Рисунок 7 – Получение AI-рекомендации

2.7 Проектирование публичного API

Публичный API разделён на группы по предметным областям:

Таблица 7 – Группы публичного API CoachLink

Группа	Назначение
`/api/v1/auth/*`	Регистрация, вход, обновление и отзыв токенов
`/api/v1/users/*`	Профиль текущего пользователя и поиск
`/api/v1/connections/*`	Заявки и связи тренер-спортсмен
`/api/v1/groups/*`	Тренировочные группы
`/api/v1/training/*`	Планы, назначения, отчёты, шаблоны
`/api/v1/notifications/*`	Уведомления
`/api/v1/analytics/*`	Аналитика тренировок
`/api/v1/ai/*`	AI-рекомендации

Такое разделение упрощает маршрутизацию на уровне API Gateway и делает API понятным для внешних клиентов.

2.8 Расширяемость под другие виды спорта

Хотя первая версия ориентирована на лёгкую атлетику, архитектура не ограничивается этим видом спорта. Универсальными являются:

- пользователи и роли;
- связи тренер-спортсмен;
- тренировочные группы;
- планы и назначения;
- отчёты;
- уведомления;
- аналитические агрегаты;
- API Gateway и межсервисное взаимодействие.

Для поддержки других видов спорта потребуется расширить предметную часть Training Service. Например, можно добавить таблицы упражнений, структурированный состав тренировочного плана, тип спорта, параметры подходов, повторений, веса, темпа, времени отдыха или других показателей. Такая доработка не требует изменения общей архитектуры: она расширяет модель тренировок, но не отменяет существующие сервисы и

сценарии.

2.9 Выводы по главе 2

В проектной части была предложена микросервисная backend-архитектура CoachLink. Архитектура включает API Gateway, сервис авторизации, сервис пользователей, сервис тренировок, сервис уведомлений, сервис аналитики и AI-сервис рекомендаций. Для хранения данных используется PostgreSQL, для событийного взаимодействия — NATS JetStream, для локального развёртывания — Docker Compose.

Спроектированная архитектура соответствует требованиям, сформулированным в аналитической главе. Она поддерживает ролевую модель, связи тренера и спортсмена, групповые назначения, стандартизированные отчёты, уведомления и аналитику. При этом система остаётся расширяемой: текущая реализация ориентирована на лёгкую атлетику, но базовая модель может быть адаптирована для других видов спорта.

3 Реализация и тестирование

3.1 Общая структура реализации

Backend-платформа CoachLink реализована как набор Go-сервисов в монорепозитории. Каждый сервис имеет собственную структуру проекта, включающую точку входа, конфигурацию, HTTP-обработчики, бизнес-логику, модели данных и слой доступа к данным или внешним клиентам.

Основные директории backend:

Таблица 8 – Основные директории backend-платформы

Директория	Назначение
`services/api-gateway`	Единая точка входа и reverse proxy
`services/auth-service`	Авторизация и управление токенами
`services/user-service`	Профили, связи и группы
`services/training-service`	Планы, назначения, отчёты, шаблоны
`services/notification-service`	Уведомления и FCM-токены
`services/analytics-service`	Аналитика тренировочных данных
`services/ai-service`	AI-рекомендации
`pkg/events`	Общие типы событий NATS
`deployments`	Docker Compose и инициализация БД
`tests/integration`	Интеграционные тесты через API Gateway

Сервисы запускаются в Docker Compose вместе с PostgreSQL, NATS JetStream, Ollama и вспомогательными контейнерами. Для каждого сервиса предусмотрен Dockerfile. Stateful-сервисы применяют SQL-миграции при запуске.

3.2 API Gateway

API Gateway реализован как отдельный Go-сервис. Его основная задача — принять внешний HTTP-запрос и перенаправить его в нужный

backend-сервис. Для этого используются префиксы маршрутов:

Таблица 9 – Маршрутизация API Gateway по префиксам

Префикс	Целевой сервис
`/api/v1/auth/*`	Auth Service
`/api/v1/users/*`	User Service
`/api/v1/connections/*`	User Service
`/api/v1/groups/*`	User Service
`/api/v1/training/*`	Training Service
`/api/v1/notifications/*`	Notification Service
`/api/v1/analytics/*`	Analytics Service
`/api/v1/ai/*`	AI Service

Gateway выполняет JWT-проверку для всех защищённых маршрутов. Исключениями являются регистрация, вход, обновление токена и health-check. После успешной проверки токена Gateway извлекает из claims идентификатор пользователя, роль и логин. Эти значения передаются дальше через заголовки:

```
X-User-ID: <user_id>
X-User-Role: <coach|athlete>
X-User-Login: <login>
```

Такой механизм позволяет внутренним сервисам принимать решения о доступе без повторной проверки JWT. Например, Training Service проверяет, может ли текущий пользователь создать план, отправить отчёт или получить назначение.

Gateway также отвечает за CORS и health-check:

```
GET /health
```

Важно отметить, что rate limiting в текущей реализации не рассматривается как реализованная функциональность. Его можно добавить в дальнейшем как middleware уровня Gateway.

3.3 Auth Service

Auth Service отвечает за регистрацию, вход и управление токенами. Сервис использует PostgreSQL-базу `auth_db` и таблицы `users` и `refresh_tokens`.

3.3.1 Регистрация

При регистрации пользователь передаёт логин, email, пароль, ФИО и роль. DTO запроса содержит ограничения:

- логин обязателен, длина от 3 до 50 символов;
- email должен иметь корректный формат;
- пароль обязателен, длина от 8 до 128 символов;
- ФИО обязательно;
- роль должна быть `coach` или `athlete`.

Дополнительно сервис проверяет формат логина регулярным выражением:

```
^[a-zA-Z0-9-]+$
```

Это ограничение исключает пробелы, кириллицу и специальные символы в логине, что упрощает поиск и использование логина в API.

Пароль хешируется через bcrypt. В базе хранится только `password_hash`, исходный пароль не сохраняется. После создания пользователя сервис публикует событие `coachlink.user.registered`, содержащее идентификатор пользователя, логин, ФИО, email и роль.

3.3.2 Вход и токены

При входе сервис ищет пользователя по логину и сравнивает пароль с bcrypt-хешем. При успешной проверке создаётся пара токенов:

- access token — JWT с коротким сроком жизни;
- refresh token — opaque token с более длинным сроком жизни.

Access token содержит claims:

```
{
  "sub": "uuid",
  "login": "athlete-login",
  "role": "athlete",
  "exp": 1234567890,
  "iat": 1234567890
}
```

Refresh token не хранится в открытом виде. Сервис вычисляет SHA-256 хеш токена и сохраняет только хеш. При refresh старый токен удаляется, затем создаётся новая пара access/refresh. При logout refresh token удаляется из базы.

3.3.3 Основные API Auth Service

Таблица 10 – Основные API Auth Service

Метод	Endpoint	Назначение
`POST`	`/api/v1/auth/register`	Регистрация
`POST`	`/api/v1/auth/login`	Вход
`POST`	`/api/v1/auth/refresh`	Обновление access token
`POST`	`/api/v1/auth/logout`	Отзыв refresh token

3.4 User Service

User Service хранит пользовательские профили, связи тренер-спортсмен и тренировочные группы. Он использует базу `user_db`.

3.4.1 Синхронизация профилей

Профиль пользователя создаётся не напрямую при регистрации, а через событие `coachlink.user.registered`. Auth Service публикует событие после успешной регистрации, а User Service подписан на него через NATS JetStream. При получении события User Service создаёт запись в таблице `user_profiles`.

Такое разделение позволяет Auth Service отвечать только за учётные данные и токены, а User Service — за доменную информацию, используемую в тренировочном процессе.

3.4.2 Поиск пользователей

Сервис предоставляет поиск по ФИО и логину. Поиск используется, например, когда спортсмен ищет тренера для отправки заявки или тренер ищет спортсмена. Минимальная длина поискового запроса — два символа. Это ограничение уменьшает количество слишком широких запросов.

Для ускорения поиска в базе включено расширение `pg_trgm`, а для полей имени и логина создаются trigram-индексы. Это подходит для поиска подстрок и автодополнения.

3.4.3 Заявки и связи

Связь тренера и спортсмена создаётся через заявку. Основной сценарий:

1. Спортсмен находит тренера.
2. Спортсмен отправляет заявку.
3. Тренер получает уведомление.
4. Тренер принимает или отклоняет заявку.
5. При принятии создаётся запись в `coach_athlete_relations`.

В первой версии у спортсмена может быть один тренер. Это ограничение задаётся уникальностью `athlete_id` в таблице связей. Такое решение упрощает логику доступа: Training Service и Analytics Service могут однозначно определять, какой тренер отвечает за спортсмена.

3.4.4 Тренировочные группы

Тренер может создавать тренировочные группы и добавлять туда своих спортсменов. Группа используется для массового назначения тренировочного плана. Перед добавлением спортсмена в группу сервис проверяет, есть ли связь между этим спортсменом и тренером.

User Service предоставляет следующие операции:

– создание группы;
– получение списка групп;
– получение деталей группы с участниками;
– обновление названия группы;
– удаление группы;
– добавление спортсмена;
– удаление спортсмена.

3.4.5 Внутренние API

User Service предоставляет внутренние API для других сервисов:

Таблица 11 – Внутренние API User Service

Endpoint	Использование
GET `/internal/users/{userId}`	Получение профиля пользователя
GET `/internal/groups/{groupId}/members`	Получение состава группы

Таблица 11 – Продолжение таблицы 11

Endpoint	Использование
GET `/internal/coach/{coachId}/has-athlete/{athleteId}`	Проверка связи тренера и спортсмена

Эти API не предназначены для внешних клиентов. Они позволяют Training Service, Analytics Service и AI Service выполнять предметные проверки и получать необходимые данные.

3.5 Training Service

Training Service является центральным сервисом тренировочного процесса. Он использует базу `training_db`.

3.5.1 Тренировочные планы

Тренировочный план содержит название, описание, дату и идентификатор тренера. План может быть назначен:

- одному или нескольким спортсменам по списку `athlete_ids`;
- всем спортсменам группы через `group_id`;
- комбинации группы и отдельных спортсменов.

Если указан `group_id`, Training Service обращается к User Service и получает участников группы. Затем создаётся один тренировочный план и набор назначений для каждого спортсмена.

3.5.2 Назначения

Назначение представляет связь между тренировочным планом и конкретным спортсменом. В назначении хранятся:

- `plan_id`;
- `athlete_id`;
- `coach_id`;
- имя и логин спортсмена;
- имя и логин тренера;
- статус;
- время назначения;

– время выполнения;
– время архивации.

Денормализация имени и логина спортсмена используется для удобной фильтрации списков назначений без дополнительного запроса в User Service.

Статусы назначений:

Таблица 12 – Статусы тренировочных назначений

Статус	Значение
`assigned`	Задание назначено и ожидает отчёта
`completed`	Спортсмен отправил отчёт
`archived`	Тренер перенёс выполненное назначение в архив

3.5.3 Просроченность задания

Просроченность вычисляется функцией `ComputeIsOverdue`. Задание считается просроченным, если:

– статус равен `assigned`;
– дата тренировки уже прошла.

Просроченность не хранится в базе данных как отдельное поле. Это уменьшает количество фоновых задач и исключает ситуацию, когда сохранённое значение устаревает.

3.5.4 Отчёты

Спортсмен может отправить отчёт только по своему назначению и только если назначение находится в статусе `assigned`. После создания отчёта статус назначения меняется на `completed`.

Формат отчёта:

```
1 {
2   "content": Тренировка" прошла хорошо, последние интервалы дались
    тяжело",
3   "duration_minutes": 75,
4   "perceived_effort": 7,
5   "max_heart_rate": 188,
6   "avg_heart_rate": 162,
7   "distance_km": 12.4
8 }
```

Поля отчёта выбраны с учётом первой реализации для лёгкой атлетики. Они позволяют фиксировать как объективные показатели, так и субъективную оценку нагрузки.

3.5.5 Шаблоны тренировок

Тренер может создавать шаблоны тренировок. Шаблон содержит название и описание. Он используется для повторяющихся тренировок, чтобы тренеру не приходилось каждый раз вручную вводить одно и то же задание. При создании плана можно сохранить его как шаблон или использовать существующий шаблон как основу.

3.5.6 События Training Service

Training Service публикует события:

Таблица 13 – События Training Service

Событие	Когда публикуется
`coachlink.training.assigned`	Создано назначение
`coachlink.training.deleted`	Назначение удалено
`coachlink.report.submitted`	Спортсмен отправил отчёт

Эти события используются Notification Service для создания уведомлений.

3.6 Notification Service

Notification Service подписан на события NATS JetStream и создаёт уведомления в базе `notification_db`.

3.6.1 Обработка событий

Для каждого типа события создаётся отдельный durable consumer. Это позволяет сервису продолжить обработку после перезапуска и не потерять события.

Примеры соответствия событий и уведомлений:

Таблица 14 – Соответствие событий и создаваемых уведомлений

Событие	Получатель	Тип уведомления
`connection.requested`	Тренер	Новая заявка от спортсмена
`connection.accepted`	Спортсмен	Заявка принята
`training.assigned`	Спортсмен	Новое тренировочное задание
`report.submitted`	Тренер	Новый отчёт
`group.athlete_added`	Спортсмен	Добавление в группу

Если уведомление успешно создано, сообщение подтверждается через `Ack`. Если произошла ошибка, сообщение отклоняется через `Nak`, чтобы оно могло быть обработано повторно.

3.6.2 API уведомлений

Notification Service предоставляет:

– получение списка уведомлений с пагинацией;
– фильтрацию по признаку прочтения;
– получение количества непрочитанных уведомлений;
– пометку одного уведомления прочитанным;
– пометку всех уведомлений прочитанными;
– регистрацию FCM-токена устройства.

FCM-интеграция реализована как опциональная. Если credentials не настроены, push-доставка отключается, но серверные уведомления продолжают работать. Это важно для локальной разработки и тестирования без внешних Firebase-настроек.

3.7 Analytics Service

Analytics Service не хранит собственные данные. Он получает отчёты и агрегаты из Training Service через внутренние HTTP API.

3.7.1 Сводка спортсмена

Сводка включает:

– количество отчётов;

– суммарную длительность;
– среднюю длительность;
– суммарную дистанцию;
– среднюю воспринимаемую нагрузку;
– средний пульс;
– максимальный пульс;
– количество назначений;
– процент выполнения.

Эти показатели позволяют тренеру оценить объём и регулярность тренировок спортсмена.

3.7.2 Прогресс по периодам

Analytics Service группирует отчёты по неделям или месяцам. Для каждого периода рассчитываются:

– количество отчётов;
– суммарная длительность;
– средняя нагрузка;
– средний пульс;
– суммарная дистанция.

Группировка по периодам позволяет видеть динамику тренировочного процесса, а не только итоговые значения за всё время.

3.7.3 Обзор тренера

Для тренера сервис формирует обзор по его спортсменам: количество спортсменов, назначений, отчётов и процент выполнения. Такой endpoint нужен для быстрого понимания общей картины по группе.

3.8 AI Service

AI Service предоставляет одну публичную ручку:

```
POST /api/v1/ai/athletes/{id}/recommendations
```

3.8.1 Проверка доступа

Сервис доступен только пользователю с ролью `coach`. После получения запроса AI Service обращается к User Service и проверяет, связан ли указанный

спортсмен с текущим тренером. Если связи нет, возвращается ошибка доступа.

3.8.2 Получение данных

После проверки доступа сервис получает из Analytics Service:

– сводную статистику спортсмена;
– последние отчёты спортсмена.

Если отчётов нет, сервис возвращает ошибку недостатка данных. Это необходимо, потому что AI-рекомендация должна опираться на фактическую тренировочную историю, а не на пустой контекст.

3.8.3 Формирование prompt

Prompt формируется на русском языке. Он содержит:

– общую статистику спортсмена;
– последние тренировки;
– длительность;
– дистанцию;
– субъективную нагрузку;
– показатели пульса;
– комментарии спортсмена;
– дополнительный контекст от тренера.

Количество отчётов ограничивается пятью. Типовой сценарий — рекомендации по последним 3-5 отчётам. Это снижает размер prompt и ускоряет генерацию.

3.8.4 Генерация через Ollama

AI Service обращается к Ollama API:

```
POST /api/chat
```

Используется модель `gemma3:4b`. Ответ возвращается тренеру в виде структуры:

```
{
  "athlete_id": "uuid",
  "type": "recommendations",
  "content": Текст" рекомендаций от LLM...",
  "generated_at": "2026-04-13T12:00:00Z",
  "model": "gemma3:4b"
```

```
}
```

AI-модуль не является заменой тренеру. Он предоставляет вспомогательный текст, который тренер может использовать при принятии решения о корректировке нагрузки.

3.9 Примеры API-запросов

3.9.1 Регистрация пользователя

```
{
  "login": "runner-coach",
  "email": "coach@example.com",
  "password": "strong-password",
  "full_name": Иванов" Иван Иванович",
  "role": "coach"
}
```

3.9.2 Создание тренировочного плана

```
{
  "title": Интервальная" тренировка",
  "description": "6x400 м через 200 м трусцой, разминка 15 минут,
   заминка 10 минут",
  "scheduled_date": "2026-04-30",
  "athlete_ids": ["athlete-uuid"],
  "save_as_template": true
}
```

3.9.3 Отправка отчёта

```
{
  "content": Выполнил" все отрезки, последние два были тяжёлыми",
  "duration_minutes": 70,
  "perceived_effort": 8,
  "max_heart_rate": 190,
  "avg_heart_rate": 165,
  "distance_km": 11.2
}
```

3.9.4 Получение AI-рекомендации

```
{
  "context": Подготовка" к старту на 800 м через 2 месяца"
}
```

3.10 Тестирование

3.10.1 Уровни тестирования

Для проверки backend используются несколько уровней тестирования.

Unit-тесты проверяют бизнес-логику отдельных сервисов без запуска всей инфраструктуры. Такой подход позволяет быстро проверять правила доступа, обработку ошибок и вычисления.

E2E smoke-тест проверяет основной пользовательский сценарий через HTTP API. Он нужен для быстрой проверки того, что основные сервисы работают совместно.

Интеграционные тесты проверяют endpoint-группы через API Gateway. Они ближе всего к поведению реального внешнего клиента, потому что запросы проходят через тот же входной слой, что и мобильное приложение.

3.10.2 Unit-тесты

Unit-тесты покрывают:

Таблица 15 – Покрытие unit-тестами по сервисам

Сервис	Что проверяется
`training-service`	Просроченность, доступ к заданиям, архивация, отправка отчётов, шаблоны
`notification-service`	Регистрация FCM-токена, пагинация, отметка прочтения, создание уведомления
`analytics-service`	Сводка, группировка по неделям/месяцам, сортировка, ошибки клиента

Таблица 15 – Продолжение таблицы 15

Сервис	Что проверяется
`ai-service`	Успешная генерация, недостаток данных, ошибки Ollama, лимит отчётов, prompt

Команда запуска:

```
make test-unit
```

Фактически подтверждённый результат: команда `make test-unit` выполнена успешно. Прошли unit-тесты `ai-service`, `analytics-service`, `training-service` и `notification-service`.

3.10.3 Компиляция интеграционных тестов

Для проверки актуальности интеграционного тестового пакета используется компиляция тестового бинарника:

```
cd tests/integration
go test -c
```

Фактически подтверждённый результат: интеграционный пакет компилируется. Это подтверждает, что тестовые функции согласованы с текущим набором endpoint-ов, в том числе с единственной реализованной AI-ручкой рекомендаций.

3.10.4 E2E и интеграционные тесты

E2E и интеграционные тесты выполнялись в поднятом окружении платформы:

```
make up
make test-e2e
make test-integration
```

Фактически подтверждённый результат: команды `make test-e2e` и `make test-integration` выполнены успешно.

3.10.5 Таблица результатов

Таблица 16 – Результаты тестовых проверок

Проверка	Статус	Комментарий
`make test-unit`	Успешно	Проверена бизнес-логика четырёх сервисов
`go test -c` в `tests/integration`	Успешно	Интеграционный пакет компилируется
`make test-e2e`	Успешно	Проверен сквозной пользовательский сценарий
`make test-integration`	Успешно	Подтверждена корректность интеграционных сценариев через API Gateway

3.11 Выводы по главе 3

В третьей главе была рассмотрена реализация backend-платформы CoachLink. Реализованы сервисы, обеспечивающие регистрацию и авторизацию пользователей, управление профилями и связями, тренировочные группы, планы, назначения, отчёты, уведомления, аналитику и AI-рекомендации.

Система построена вокруг единого API Gateway, который проверяет JWT и маршрутизирует запросы. Межсервисное взаимодействие реализовано через HTTP и NATS JetStream. Данные разделены по сервисам, что соответствует выбранной микросервисной архитектуре.

Проведённая проверка показывает, что ключевая бизнес-логика покрыта unit-тестами, а сценарии уровня E2E и integration успешно выполняются при поднятой платформе. Это подтверждает работоспособность backend-контуров как на уровне отдельных сервисов, так и на уровне межсервисного взаимодействия через API Gateway.

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы была спроектирована и реализована backend-часть открытой микросервисной платформы CoachLink, предназначенной для автоматизации взаимодействия в системе «тренер-спортсмен». Разработанная система решает практическую проблему разрозненного обмена тренировочными планами и отчетами через мессенджеры, электронные таблицы и произвольные файлы. Вместо набора не связанных между собой каналов коммуникации платформа предоставляет единый backend-контур, в котором тренировочные задания, отчеты, уведомления и аналитические данные хранятся и обрабатываются в структурированном виде.

Основной результат работы состоит в создании серверной основы платформы. Мобильное приложение в рамках данной работы рассматривается только как внешний клиент REST API и не является результатом разработки автора. Такой фокус позволяет рассматривать систему именно с позиции backend-проектирования: разделения ответственности между сервисами, организации API, хранения данных, событийного взаимодействия, авторизации, тестирования и возможности дальнейшего развития.

Достижение цели работы

Целью работы была разработка открытой микросервисной backend-платформы для стандартизации и автоматизации взаимодействия между тренерами и спортсменами. Данная цель была достигнута за счет проектирования и реализации набора независимых сервисов, каждый из которых отвечает за отдельную область предметной логики.

В платформе реализованы следующие ключевые возможности:

- регистрация и аутентификация пользователей с использованием JWT;
- разделение пользователей по ролям тренера и спортсмена;
- хранение пользовательских профилей;
- оформление и обработка заявок на установление связи между тренером и спортсменом;
- создание тренировочных групп;
- создание тренировочных планов и шаблонов;

– назначение тренировок отдельным спортсменам и группам;
– отправка спортсменами стандартизированных отчетов о выполнении тренировок;
– хранение статусов назначений и отчетов;
– формирование уведомлений на основе событий системы;
– агрегация тренировочных данных для аналитических представлений;
– получение AI-рекомендаций по последним отчетам спортсмена.

Таким образом, backend-платформа охватывает полный цикл базового взаимодействия: от регистрации пользователя и установления связи между тренером и спортсменом до назначения тренировки, отправки отчета, уведомления участников и последующего анализа данных.

Решение поставленных задач

В ходе выполнения работы были решены задачи, сформулированные во введении.

Первая задача заключалась в анализе предметной области и существующих способов организации тренировочного процесса. Было показано, что мессенджеры, таблицы, спортивные трекеры и закрытые платформы решают отдельные части проблемы, но не предоставляют одновременно единый формат отчетности, поддержку ролей тренера и спортсмена, групповые назначения, открытую архитектуру, API и возможность самостоятельного развертывания. На основе этого анализа были сформулированы требования к backend-платформе.

Вторая задача заключалась в проектировании архитектуры системы. Для решения этой задачи была выбрана микросервисная архитектура, в которой функциональность разделена между API Gateway, Auth Service, User Service, Training Service, Notification Service, Analytics Service и AI Service. Такое разделение позволяет развивать отдельные части системы независимо и снижает связанность между модулями. Для межсервисного взаимодействия используются REST API и событийная коммуникация через NATS JetStream.

Третья задача состояла в реализации сервисов платформы. В рамках backend-части были реализованы сервисы аутентификации, пользователей, тренировок, уведомлений, аналитики, AI-рекомендаций и API Gateway. Каждый сервис имеет собственную область ответственности и собственную

схему данных либо, в случае stateless-сервисов, работает как вычислительный модуль поверх данных других сервисов.

Четвертая задача была связана с описанием публичного API и межсервисных контрактов. Платформа предоставляет REST API, сгруппированный по функциональным областям: аутентификация, пользователи, тренировки, уведомления, аналитика и AI-рекомендации. API Gateway выступает единой точкой входа и отвечает за проверку JWT, маршрутизацию запросов и передачу пользовательского контекста во внутренние сервисы.

Пятая задача заключалась в проверке работоспособности backend-части. Для проекта используются unit-тесты сервисного слоя, E2E smoke-сценарий и интеграционные тесты через API Gateway. Подтверждены успешные запуски unit-тестов сервисов, компиляции интеграционного тестового пакета, а также E2E и полного интеграционного прогона при поднятой платформе.

Практическая значимость

Практическая значимость работы заключается в том, что разработанный backend может использоваться как основа для реального сервиса взаимодействия тренера и спортсмена. Платформа устраняет несколько типичных проблем, возникающих в тренировочном процессе.

Во-первых, тренер получает единый механизм назначения тренировок. Вместо отправки сообщений каждому спортсмену отдельно он может создавать планы, назначать их конкретным спортсменам или группам и отслеживать статусы выполнения.

Во-вторых, спортсмен получает единый формат отчета. Это особенно важно для последующего анализа, поскольку структурированные данные можно агрегировать, сравнивать и использовать для построения статистики. В мессенджерах данные обычно существуют в виде произвольного текста, фотографий или файлов, что затрудняет автоматическую обработку.

В-третьих, платформа формирует централизованную историю тренировочного процесса. Накопленные данные позволяют анализировать динамику нагрузки, регулярность выполнения тренировок и изменения показателей спортсмена.

В-четвертых, система уведомлений снижает вероятность пропуска важных событий. Пользователь может получать уведомления о новых заявках,

назначениях, отчетах и других действиях, связанных с тренировочным процессом.

В-пятых, AI Service добавляет вспомогательный аналитический слой. Он не заменяет тренера и не принимает решений вместо него, но помогает сформировать рекомендации на основе последних отчетов спортсмена. Такой подход делает AI-модуль безопасным с точки зрения роли в системе: итоговое решение остается за специалистом.

Архитектурные результаты

Одним из важных результатов работы является то, что платформа построена как набор независимых сервисов, а не как монолитное приложение. Это решение оправдано предметной областью, поскольку разные части системы имеют разные причины для изменения.

Auth Service отвечает за регистрацию, вход, обновление токенов и публикацию события `coachlink.user.registered`. Он изолирует логику безопасности и не смешивает ее с предметной логикой тренировок.

User Service отвечает за профили, заявки, связи между тренерами и спортсменами, группы и внутренние проверки доступа. Он является источником информации о социальных связях внутри платформы.

Training Service отвечает за тренировочные планы, шаблоны, назначения и отчеты. Это центральный предметный сервис, в котором сосредоточена основная логика тренировочного процесса.

Notification Service подписывается на события из NATS JetStream и создает уведомления для пользователей. Благодаря этому сервисы, в которых возникают события, не обязаны напрямую знать о способе доставки уведомлений.

Analytics Service выполняет stateless-агрегацию тренировочных данных. Он получает данные из Training Service и формирует summary, progress и coach overview без хранения собственной предметной базы.

AI Service предоставляет одну реализованную ручку рекомендаций по последним отчетам спортсмена. Он использует данные аналитики и генерацию через Ollama, оставаясь вспомогательным модулем.

API Gateway объединяет сервисы за единым публичным API, проверяет JWT, проксирует запросы и передает во внутренние сервисы пользовательский контекст.

Такое разделение повышает сопровождаемость системы: изменение механизма уведомлений не требует переписывания Training Service, изменение аналитики не влияет на регистрацию пользователей, а развитие AI-модуля может происходить отдельно от базового цикла назначения и отчетности.

Ограничения текущей версии

Текущая версия платформы имеет осознанные ограничения, связанные с рамками дипломной работы и реалистичным объемом реализации.

Предметная реализация ориентирована на легкую атлетику. Это сделано потому, что автор занимается этим видом спорта и лучше понимает структуру тренировочных данных: дистанции, объем, интенсивность, самочувствие, комментарии по выполнению. При этом архитектура не ограничивает платформу только легкой атлетикой. Поддержка других видов спорта может быть реализована через развитие структуры тренировок, отчетов, шаблонов и sport-specific метрик.

Мобильное приложение не входит в зону ответственности данной работы. Backend предоставляет REST API, к которому могут подключаться разные клиенты: мобильное приложение, web-интерфейс, административные панели или внешние интеграции.

AI Service в текущей версии не является универсальной системой анализа тренировочного процесса. Он реализует один сценарий: формирование рекомендаций по последним отчетам конкретного спортсмена. Расширенные AI-сценарии, такие как комплексный анализ периода или сводка по группе для тренера, относятся к направлениям дальнейшего развития.

В текущем backend не реализованы production-механизмы уровня промышленной эксплуатации: rate limiting, Redis-кеширование, расширенная наблюдаемость, автоматическое масштабирование, полноценная отказоустойчивость и интеграции со спортивными устройствами. Эти направления относятся к дальнейшему развитию.

Направления дальнейшего развития

Дальнейшее развитие платформы может идти по нескольким направлениям.

Первое направление - поддержка других видов спорта. Для этого необходимо расширить модель тренировочного плана и отчета так, чтобы она позволяла описывать не только беговые и легкоатлетические тренировки, но и силовые упражнения, игровые тренировки, плавание, велоспорт и другие виды активности. Важно сохранить общую часть модели: назначение, отчет, статус, комментарий, связь тренера и спортсмена, но добавить специализированные поля для отдельных спортивных дисциплин.

Второе направление - более детальная структура тренировок. В текущей версии тренировка может описываться как план и отчет, но в будущем можно выделить отдельные упражнения, интервалы, серии, повторения, зоны интенсивности, контрольные показатели и целевые метрики. Это позволит точнее сравнивать план и факт выполнения.

Третье направление - интеграции со спортивными устройствами и внешними сервисами. Платформа может получать данные из часов, пульсометров, GPS-трекеров и популярных спортивных экосистем. Это позволит автоматически заполнять часть отчета и снижать нагрузку на спортсмена.

Четвертое направление - развитие аналитики. Можно добавить анализ нагрузки по периодам, выявление пропусков, сравнение планового и фактического объема, оценку восстановления, предупреждения о резком росте нагрузки и индивидуальные графики прогресса.

Пятое направление - расширение AI-сценариев. После накопления большего объема данных можно реализовать дополнительные функции: сводки по группе, анализ динамики спортсмена за период, выявление потенциальных рисков, подготовку черновика комментария тренера. При этом AI должен оставаться вспомогательным инструментом, а не заменой профессионального решения тренера.

Шестое направление - подготовка backend к production-эксплуатации. Сюда относятся централизованное логирование, трассировка запросов, метрики, health checks, rate limiting, управление секретами, резервное копирование баз данных, мониторинг очередей и настройка CI/CD.

Итоговый вывод

Разработанная backend-платформа CoachLink демонстрирует возможность построения открытой микросервисной системы для

автоматизации тренировочного процесса. Система объединяет роли тренера и спортсмена, стандартизирует обмен тренировочными данными, поддерживает групповые сценарии, формирует уведомления и создает основу для аналитики.

Главный результат работы заключается не только в реализации отдельных API-методов, но и в создании расширяемой серверной архитектуры. Платформа может развиваться как в сторону более глубокого описания тренировок в легкой атлетике, так и в сторону поддержки других видов спорта. Благодаря открытой архитектуре и разделению сервисов проект может быть дополнен новыми клиентскими приложениями, интеграциями, аналитическими модулями и AI-сценариями без полного перепроектирования backend-части.

Поставленная цель достигнута, а задачи выпускной квалификационной работы выполнены в полном объеме.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Richardson C. Microservices Patterns: With Examples in Java. — Shelter Island : Manning Publications, 2018. — ISBN 978-1617294549.

2. Newman S. Building Microservices: Designing Fine-Grained Systems. — 2-е изд. — Sebastopol : O'Reilly Media, 2021. — ISBN 978-1492034025.

3. Microservices: Yesterday, Today, and Tomorrow / N. Dragoni [и др.] // Present and Ulterior Software Engineering. — Cham : Springer, 2017. — С. 195—216. — DOI: 10.1007/978-3-319-67425-4_12.

4. Balalaie A., Heydarnoori A., Jamshidi P. Microservices Architecture Enables DevOps: Migration to a Cloud-Native Architecture // IEEE Software. — 2016. — Т. 33, № 3. — С. 42—52. — DOI: 10.1109/MS.2016.64.

5. The Go Programming Language Documentation / The Go Authors. — URL: https://go.dev/doc/ (дата обращения 26.04.2026).

6. Echo Web Framework Documentation / LabStack. — URL: https://echo.labstack.com/docs (дата обращения 26.04.2026).

7. PostgreSQL Documentation / PostgreSQL Global Development Group. — URL: https://www.postgresql.org/docs/ (дата обращения 26.04.2026).

8. RFC 7519: JSON Web Token (JWT) / Internet Engineering Task Force. — URL: https://www.rfc-editor.org/rfc/rfc7519 (дата обращения 26.04.2026).

9. NATS Documentation / Synadia Communications. — URL: https://docs.nats.io/ (дата обращения 26.04.2026).

10. Docker Compose Documentation / Docker Inc. — URL: https://docs.docker.com/compose/ (дата обращения 26.04.2026).

11. OpenAPI Specification / OpenAPI Initiative. — URL: https://spec.openapis.org/oas/latest.html (дата обращения 26.04.2026).

12. Ollama Documentation / Ollama. — URL: https://github.com/ollama/ollama (дата обращения 26.04.2026).

ПРИЛОЖЕНИЕ А
Состав Backend-Сервисов

В приложении приведена таблица с кратким описанием сервисов. Она позволяет наглядно показать состав backend-платформы и зону ответственности каждого сервиса.

Таблица А.1 – Состав backend-сервисов CoachLink

Сервис	Назначение	Хранилище	Основные взаимодействия
API Gateway	Единая точка входа, JWT-проверка, reverse proxy	Без собственной БД	Принимает внешние REST-запросы и маршрутизирует их на внутренние сервисы
Auth Service	Регистрация, login, refresh/logout, выдача JWT	PostgreSQL	Публикует `coachlink.user.registered` в NATS
User Service	Профили, заявки, связи, группы, поиск	PostgreSQL	Принимает событие регистрации, предоставляет внутренние проверки доступа
Training Service	Планы, шаблоны, назначения, отчеты, статусы	PostgreSQL	Публикует события назначений и отчетов, предоставляет данные аналитике
Notification Service	Уведомления, unread count, FCM-токены	PostgreSQL	Подписывается на события NATS
Analytics Service	Summary, progress, coach overview	Stateless	Получает данные из Training Service и User Service
AI Service	Рекомендации по последним отчетам спортсмена	Stateless	Проверяет связь через User Service, получает данные из Analytics Service, обращается к Ollama

AI Service и Analytics Service в текущей версии реализованы как stateless-сервисы. Они не владеют собственными предметными данными и формируют результат на основе данных других сервисов.

ПРИЛОЖЕНИЕ Б
Группы Публичного API

В приложении представлена таблица публичных API-групп. Полный перечень endpoints содержится в `docs/api/openapi.yaml`, а в тексте диплома используется сгруппированное представление.

Таблица Б.1 – Группы публичного API и их назначение

Группа API	Примеры endpoints	Назначение
Auth API	POST `/api/v1/auth/register`, POST `/api/v1/auth/login`	Регистрация и аутентификация пользователей
User API	`/api/v1/users/me`, `/api/v1/users/search`	Профили, поиск, заявки, связи, группы
Training API	`/api/v1/training/plans`, `/api/v1/training/assignments/{id}/report`	Планы, назначения, отчеты и шаблоны
Notification API	`/api/v1/notifications`, `/api/v1/notifications/{id}/read`	Уведомления и отметка прочтения
Analytics API	`/api/v1/analytics/athletes/{id}/summary`, `/api/v1/analytics/coach/overview`	Агрегация тренировочных данных
AI API	POST `/api/v1/ai/athletes/{id}/recommendations`	Рекомендации по последним отчетам спортсмена
System API	`/health`, `/api/v1/health`	Проверка доступности сервисов

В дипломе для AI Service указана только реализованная ручка рекомендаций.

ПРИЛОЖЕНИЕ В
Примеры JSON-Запросов И Ответов

Примеры API отражают backend-логику и не зависят от особенностей мобильного интерфейса. В качестве репрезентативных примеров используются отправка отчета и получение AI-рекомендации.

Пример Отправки Отчета О Тренировке

```
1 {
2   "content": Выполнил" кросс спокойно, последние 2 км немного
    быстрее.",
3   "duration_minutes": 46,
4   "perceived_effort": 6,
5   "max_heart_rate": 174,
6   "avg_heart_rate": 151,
7   "distance_km": 8.0
8 }
```

Пример показывает тело запроса для endpoint `POST /api/v1/training/assignments/{assignmentId}/report`. Идентификатор назначения передается в path-параметре, а отчет хранится в структурированном виде. В отличие от сообщения в мессенджере, backend может использовать поля отчета для аналитики: считать недельный объем, длительность, дистанцию, среднюю воспринимаемую нагрузку и пульсовые показатели.

Пример Ответа На Создание Отчета

```
1 {
2   "id": "7f9ef6c3-0fd9-4cfd-a69d-d3d1b65a157d",
3   "assignment_id": "2b54c487-0ef3-4e60-8e9a-7f2fdb66c101",
4   "athlete_id": "a52d4f18-e218-4d2a-bd4f-01bcb89d01f2",
5   "athlete_full_name": Петров" Иван Сергеевич",
6   "athlete_login": "ivan-petrov",
7   "content": Выполнил" кросс спокойно, последние 2 км немного
    быстрее.",
8   "duration_minutes": 46,
9   "perceived_effort": 6,
10  "max_heart_rate": 174,
```

```
  "avg_heart_rate": 151,
  "distance_km": 8.0,
  "created_at": "2026-04-18T18:30:00Z"
}
```

После создания отчета Training Service может опубликовать событие, на которое подпишется Notification Service. Благодаря этому тренер получает уведомление без прямой зависимости между Training Service и способом доставки уведомлений.

Пример Запроса AI-Рекомендации

```
{
  "context": Подготовка" к старту на 800 м через два месяца."
}
```

Тело запроса является необязательным. Поле `context` позволяет тренеру передать дополнительный контекст, например информацию о предстоящем старте. Количество отчетов не передается в запросе: сервис автоматически использует последние отчеты спортсмена и в текущей реализации ограничивает выборку пятью записями. Типовым сценарием является формирование рекомендаций по 3-5 последним тренировкам.

Пример Ответа AI Service

```
{
  "athlete_id": "a52d4f18-e218-4d2a-bd4f-01bcb89d01f2",
  "type": "recommendations",
  "content": Тенденции"**** — спортсмен стабильно выполняет объем,
   но воспринимаемая нагрузка растет. Рекомендации**** — оставить
   один восстановительный день или снизить интенсивность следующей
   тренировки.",
  "generated_at": "2026-04-18T18:35:00Z",
  "model": "gemma3:4b"
}
```

AI-рекомендация является вспомогательной информацией. Она не заменяет решение тренера и не используется как автоматическое назначение тренировочного плана.

ПРИЛОЖЕНИЕ Г
ER-Диаграмма Основных Таблиц

ER-диаграмма вынесена в приложение в расширенном виде.

```
erDiagram
    AUTH_USERS {
        uuid id PK
        string email
        string password_hash
        string role
        timestamp created_at
    }

    USER_PROFILES {
        uuid id PK
        string login
        string email
        string full_name
        string role
        timestamp created_at
    }

    CONNECTION_REQUESTS {
        uuid id PK
        uuid coach_id
        uuid athlete_id
        string status
        timestamp created_at
    }

    TRAINING_GROUPS {
        uuid id PK
        uuid coach_id
        string name
        timestamp created_at
    }

    GROUP_MEMBERS {
        uuid group_id
        uuid athlete_id
        timestamp added_at
```

```
38      }
39
40      TRAINING_PLANS {
41          uuid id PK
42          uuid coach_id
43          string title
44          text description
45          date scheduled_date
46          uuid group_id
47          timestamp created_at
48      }
49
50      TRAINING_ASSIGNMENTS {
51          uuid id PK
52          uuid plan_id
53          uuid athlete_id
54          uuid coach_id
55          string status
56          timestamp assigned_at
57          timestamp completed_at
58          timestamp archived_at
59      }
60
61      TRAINING_REPORTS {
62          uuid id PK
63          uuid assignment_id
64          uuid athlete_id
65          text content
66          int duration_minutes
67          int perceived_effort
68          int max_heart_rate
69          int avg_heart_rate
70          decimal distance_km
71          timestamp created_at
72      }
73
74      NOTIFICATIONS {
75          uuid id PK
76          uuid user_id
77          string type
78          string title
79          text body
```

```
80          bool is_read
81          timestamp created_at
82      }
83
84      AUTH_USERS ||--|| USER_PROFILES : synced_by_event
85      USER_PROFILES ||--o{ CONNECTION_REQUESTS : participates
86      USER_PROFILES ||--o{ TRAINING_GROUPS : owns
87      TRAINING_GROUPS ||--o{ GROUP_MEMBERS : contains
88      TRAINING_PLANS ||--o{ TRAINING_ASSIGNMENTS : produces
89      TRAINING_ASSIGNMENTS ||--o| TRAINING_REPORTS : completed_by
90      USER_PROFILES ||--o{ NOTIFICATIONS : receives
```

Диаграмма показывает логическую связь основных сущностей. В микросервисной архитектуре таблицы физически распределены по базам данных отдельных сервисов, а не размещаются в единой общей базе.

ПРИЛОЖЕНИЕ Д
Sequence Diagrams Для Ключевых Сценариев

Регистрация Пользователя

```
sequenceDiagram
    participant Client as REST Client
    participant Gateway as API Gateway
    participant Auth as Auth Service
    participant NATS as NATS JetStream
    participant User as User Service

    Client->>Gateway: POST /api/v1/auth/register
    Gateway->>Auth: proxy request
    Auth->>Auth: hash password
    Auth->>Auth: create auth user
    Auth->>NATS: publish coachlink.user.registered
    Auth-->>Gateway: JWT tokens
    Gateway-->>Client: registration response
    NATS-->>User: coachlink.user.registered
    User->>User: create profile
```

Назначение Тренировки

```
sequenceDiagram
    participant Coach as Coach Client
    participant Gateway as API Gateway
    participant Training as Training Service
    participant User as User Service
    participant NATS as NATS JetStream
    participant Notifications as Notification Service

    Coach->>Gateway: POST /api/v1/assignments
    Gateway->>Gateway: validate JWT
    Gateway->>Training: request with X-User-ID and X-User-Role
    Training->>User: check coach-athlete relation
    User-->>Training: relation exists
    Training->>Training: create assignment
    Training->>NATS: publish coachlink.training.assigned
    Training-->>Gateway: assignment
```

```
    Gateway-->>Coach: response
    NATS-->>Notifications: coachlink.training.assigned
    Notifications->>Notifications: create notification
```

Отправка Отчета

```
sequenceDiagram
    participant Athlete as Athlete Client
    participant Gateway as API Gateway
    participant Training as Training Service
    participant NATS as NATS JetStream
    participant Notifications as Notification Service

    Athlete->>Gateway: POST /api/v1/reports
    Gateway->>Gateway: validate JWT
    Gateway->>Training: request with user context
    Training->>Training: validate assignment owner
    Training->>Training: save report
    Training->>NATS: publish coachlink.report.submitted
    Training-->>Gateway: report
    Gateway-->>Athlete: response
    NATS-->>Notifications: coachlink.report.submitted
    Notifications->>Notifications: notify coach
```

Получение AI-Рекомендации

```
sequenceDiagram
    participant Coach as Coach Client
    participant Gateway as API Gateway
    participant AI as AI Service
    participant User as User Service
    participant Analytics as Analytics Service
    participant Ollama as Ollama

    Coach->>Gateway: POST /api/v1/ai/athletes/{id}/recommendations
    Gateway->>Gateway: validate JWT
    Gateway->>AI: request with coach context
    AI->>User: check coach-athlete relation
    User-->>AI: relation allowed
    AI->>Analytics: get summary and latest reports
    Analytics-->>AI: aggregated data
```

```
    AI->>AI: build Russian prompt
    AI->>Ollama: generate recommendation
    Ollama-->>AI: generated text
    AI-->>Gateway: recommendation
    Gateway-->>Coach: response
```

Диаграммы размещены в приложениях. Они описывают backend-сценарии и не зависят от конкретного мобильного интерфейса.

ПРИЛОЖЕНИЕ Е
События NATS JetStream

В приложении приведена таблица событий, используемых для слабосвязанного взаимодействия сервисов.

Таблица Е.1 – События NATS JetStream для межсервисного взаимодействия

Событие	Источник	Получатель	Назначение
`coachlink.user.registered`	Auth Service	User Service	Создание профиля пользователя
`coachlink.connection.requested`	User Service	Notification Service	Уведомление тренера о новой заявке
`coachlink.connection.accepted`	User Service	Notification Service	Уведомление спортсмена о принятии заявки
`coachlink.connection.rejected`	User Service	Notification Service	Уведомление спортсмена об отклонении заявки
`coachlink.group.athlete_added`	User Service	Notification Service	Уведомление о добавлении в группу
`coachlink.group.athlete_removed`	User Service	Notification Service	Уведомление об удалении из группы
`coachlink.training.assigned`	Training Service	Notification Service	Уведомление о назначенной тренировке
`coachlink.training.deleted`	Training Service	Notification Service	Уведомление об удалении назначения
`coachlink.report.submitted`	Training Service	Notification Service	Уведомление тренера о новом отчете

Названия событий взяты из общего пакета `pkg/events` и соответствуют текущей событийной модели платформы.

ПРИЛОЖЕНИЕ Ж
Таблица Тестирования

Таблица тестирования отражает фактически подтвержденные результаты запусков.

Таблица Ж.1 – Уровни тестирования и критерии фиксации результатов

Уровень тестирования	Команда	Назначение	Статус
Unit-тесты	make test-unit	Проверка сервисного слоя отдельных микросервисов	Успешно
Компиляция интеграционного пакета	go test -c ./tests/integration	Проверка сборки интеграционных тестов	Успешно
E2E smoke	make test-e2e	Проверка базового e2e-сценария при поднятой платформе	Успешно
Integration	make test-integration	Проверка API Gateway и межсервисных сценариев	Успешно

Проверки выполнялись в рабочем окружении проекта; результаты запусков зафиксированы в таблице тестирования.

ПРИЛОЖЕНИЕ И
Команды Быстрого Запуска

Краткий порядок запуска вынесен в приложение, а подробная инструкция приведена в `docs/quickstart.md`.

```
docker compose up -d
```

```
make test-unit
```

```
make test-e2e
```

```
make test-integration
```

В основном тексте зафиксировано, что локальное окружение поднимается через Docker Compose, а подробная инструкция приведена в документации проекта.